PREMIUM HANDWRITTEN INTERVIEW NOTEBOOK

JVM

Garbage Collectors & Performance Tuning

Understand memory. Read GC evidence. Make Java faster with measured decisions.

100 ORIGINAL QUESTIONS | 100 ANSWERED FOLLOW-UPS | 20 CHAPTERS | 10 HANDS-ON LABS

```
roots → live objects → GC
JFR + GC logs → hypothesis
baseline → experiment → prove
```

Java 17 labs | Java 21 & 25 version notes | October 2026

START HERE

Learn it. Sketch it. Measure it.

Your illustrated JVM interview notebook: the exact original questions, clear interview-ready explanations, real Java examples, and a complete answer to every follow-up.

100	100	10
Unchanged questions	New follow-up answers	Reproducible exercises

Read each page in this order: simple answer → small diagram → precise explanation → Java/command example → what really happens → follow-up answer → common pitfall.

JDK baseline: Exercises are Java 17 compatible. ZGC and Shenandoah features change on JDK 21 and 25. Always inspect the exact JDK vendor, mode and supported flags in your environment.

Verification: The seven standalone Java programs compile on JDK 17 and all ten local exercises execute in isolated folders. JMH is a template requiring its own dependencies; it has not been benchmarked here.

Safety: Exercise L09 deliberately exhausts a tiny heap in a separate process. Diagnostic recordings and heap dumps can contain sensitive data. Never run failure labs within a production service.

CONTENTS

20 chapters from memory to diagnosis

01 JVM architecture and execution

02 Class loading, linking and initialization

03 Heap, object layout and native memory

04 JIT compilation and optimization

05 Garbage collection fundamentals

06 References, cleanup and retention leaks

07 Choosing a garbage collector

08 G1 architecture and phases

09 Evidence-driven G1 tuning

10 ZGC and Shenandoah

11 Heap sizing and OutOfMemoryError

12 GC logs and quantitative interpretation

13 JFR, dumps and native-memory tools

14 CPU, locks and tail latency

15 Allocation-aware Java coding

16 Benchmarking and experiment design

17 Containers and Kubernetes JVM budgets

18 Spring Boot production tuning

19 Modern Java version traps

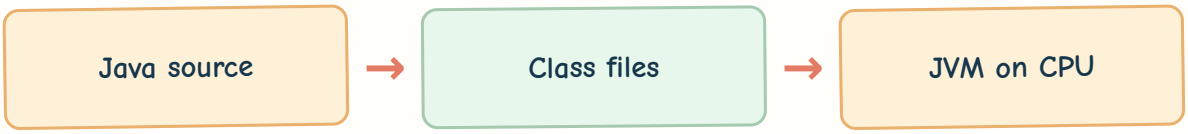
20 Senior diagnosis and design scenarios

For interviews: Chapters 1–6 establish JVM fundamentals; 7–12 explain GC and logs; 13–18 cover tools, tuning and containers; 19–20 test modern versions and senior diagnosis.

CHAPTER 01 / 20

JVM architecture and execution

Track where code runs, not only where it was written.



FIVE ORIGINAL INTERVIEW QUESTIONS

001. What is the JVM and how does it differ from the JDK?

002. What are the JVM run-time data areas?

003. What is the difference between stack and heap?

004. Does Java interpretation mean code never becomes native?

005. What is the difference between JVM behavior and OS behavior?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q001 · JVM architecture and execution

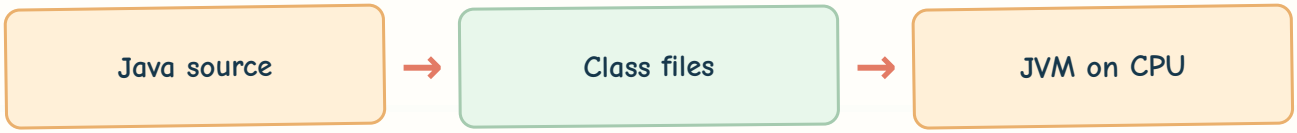
INTERVIEW ANSWER CARD

Q001. What is the JVM and how does it differ from the JDK?

SHORT ANSWER · SAY THIS FIRST

> The JVM runs Java bytecode; the JDK includes the JVM plus tools to build and diagnose Java programs.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

The JVM executes class files according to a specification. A JDK supplies development tools and a runtime implementation. HotSpot is one JVM implementation; a collector or flag described for HotSpot is not automatically portable to every JVM.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Java source -> compiler -> class files
Class files -> JVM loading and execution
JDK -> compiler, runtime and diagnostic tools
```

Why it matters: Portable bytecode runs through a platform-specific implementation.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why does the same JAR still need testing on another CPU architecture?

Answer: Bytecode is portable, but the JVM, CPU instructions, native libraries, architecture-specific images and performance can differ. Test the actual deployment architecture.

Interview trap: A minimal runtime image may omit javac, jcmd or jfr even though java works.

Source guide p. 6 · Technical references J01, J04-J06

Q002 · JVM architecture and execution

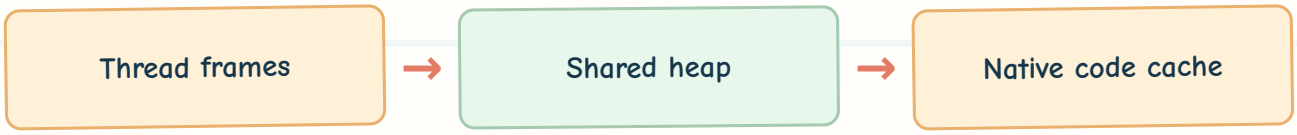
INTERVIEW ANSWER CARD

Q002. What are the JVM run-time data areas?

SHORT ANSWER · SAY THIS FIRST

> The JVM specification describes memory and execution areas shared by threads or owned by one thread.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

The specification defines per-thread PC registers and JVM stacks, a shared heap and method area, run-time constant pools and native method stacks where used. The physical implementation and memory accounting are implementation decisions.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Per thread: PC and invocation frames
Shared: heap and method-area responsibilities
HotSpot: also inspect metaspace and code cache
```

Why it matters: Specification categories describe responsibilities, not a universal native-memory layout.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Where does HotSpot keep compiled machine code?

Answer: HotSpot stores JIT-compiled instructions in a native code cache, not inside the Java object heap.

Interview trap: Equating the method area with a Java heap generation is misleading.

Source guide p. 7 · Technical references J01, J04-J06

Q003 · JVM architecture and execution

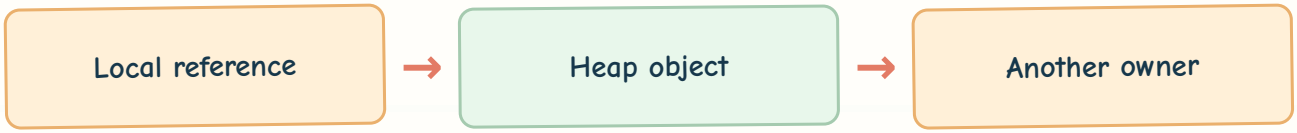
INTERVIEW ANSWER CARD

Q003. What is the difference between stack and heap?

SHORT ANSWER · SAY THIS FIRST

> Each method call has a stack frame; the heap holds objects and arrays that references can point to.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

An invocation frame contains local-variable slots and operand-stack state. The heap is the specified allocation area for objects and arrays. A reference in a local slot is not the object itself; optimizations can eliminate physical allocations without changing semantics.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Object value = new Object();
// A local reference can point to an object.
// The object can outlive this invocation if retained.
```

Why it matters: Lifetime depends on reachability, not merely lexical declaration location.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Can an object remain reachable after the creating method returns?

Answer: Yes. A returned reference, static field or another live object can still point to it, so the object stays reachable.

Interview trap: 'Every local object lives on the stack' confuses the reference with its referent.

Source guide p. 7 · Technical references J01, J04-J06

Q004 · JVM architecture and execution

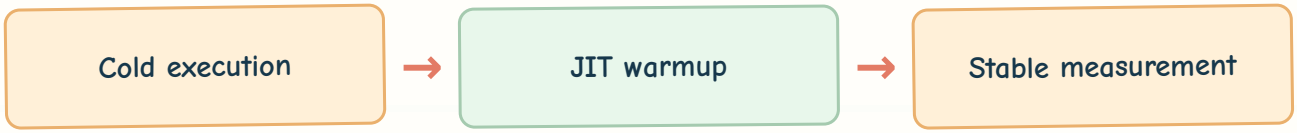
INTERVIEW ANSWER CARD

Q004. Does Java interpretation mean code never becomes native?

SHORT ANSWER · SAY THIS FIRST

> HotSpot can first interpret bytecode and later compile popular methods into fast machine code.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

No. HotSpot can interpret bytecode and compile frequently executed code into native instructions. Compilation, profiling and deoptimization operate during execution. A cold and warmed-up application can therefore show different performance.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Cold method -> interpretation and profiling
Hot path -> compiled code
Invalid assumption -> deoptimization and adaptation
```

Why it matters: The runtime can optimize using behavior observed in the running application.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why should a performance test include warmup?

Answer: Warmup allows class loading, JIT profiling and compilation to settle. Otherwise startup effects distort a steady-state benchmark.

Interview trap: 'Java is always interpreted' and 'everything compiles before startup' are both overgeneralizations.

Source guide p. 7 · Technical references J01, J04-J06

Q005 · JVM architecture and execution

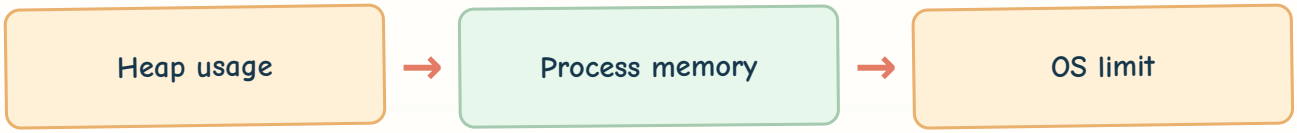
INTERVIEW ANSWER CARD

Q005. What is the difference between JVM behavior and OS behavior?

SHORT ANSWER · SAY THIS FIRST

> The JVM runs Java code and manages heap/GC; the OS schedules work and enforces process memory and CPU limits.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

The JVM manages Java execution and selected resources, while the OS schedules threads, supplies virtual memory and enforces process/container limits. A JVM metric must be interpreted together with OS evidence.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Heap used: 400 MiB
Process RSS: 750 MiB
Container limit: 1 GiB
CPU quota: independent constraint
```

Why it matters: Java heap statistics cover only one portion of process resources.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What can terminate Java without a Java OutOfMemoryError?

Answer: Linux can kill the process when a cgroup or host memory budget is exceeded. An external SIGKILL or node failure also stops Java without a Java OOME.

Interview trap: RSS minus heap-used is not a precise native-memory breakdown.

Source guide p. 8 · Technical references J01, J04-J06

CHAPTER 02 / 20

Class loading, linking and initialization

A class can exist before its static code runs.



FIVE ORIGINAL INTERVIEW QUESTIONS

006. What are loading, linking and initialization?

007. When does static initialization happen?

008. Why can the same class name represent different types?

009. How do ClassNotFoundException and NoClassDefFoundError differ?

010. How can class loaders leak memory?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q006. What are loading, linking and initialization?

SHORT ANSWER · SAY THIS FIRST

> Loading finds a class, linking checks and prepares it, and initialization runs its static setup code.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Loading obtains a class definition. Linking verifies it, prepares its static storage and resolves symbolic references as required. Initialization executes the class initialization method according to language and JVM rules. Resolution can be lazy within permitted constraints.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Load -> verify -> prepare -> resolve as needed
Initialize -> execute static field initializers/blocks
Use -> execute ordinary program behavior
```

Why it matters: These stages explain different errors and side effects.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Can a class be loaded without running its static initializer?

Answer: Yes. Class.forName(name, false, loader) can load without initializing; initialization waits for a qualifying active use.

Interview trap: Class loading and class initialization are not synonyms.

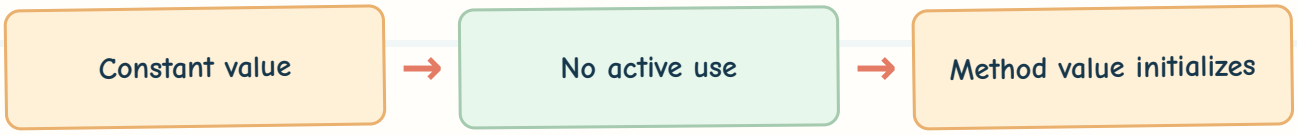
Source guide p. 9 · Technical references J02, J03, J24

Q007. When does static initialization happen?

SHORT ANSWER · SAY THIS FIRST

> Static initialization runs at a Java-defined active-use event, not whenever a class name appears.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Initialization is triggered by specified active uses, such as certain static member access, construction and reflection. Access to a compile-time constant can avoid initializing its declaring class. Initialization ordering follows superclass and relevant interface rules.

JAVA / COMMAND / SCENARIO EXAMPLE

```
class Config {
 static final int PORT = 8080; // constant variable
 static { System.out.println("init"); }
}
// Reading Config.PORT alone need not print init.
```

Why it matters: The compiler can embed the constant into caller bytecode.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What changes if PORT is assigned by a method call?

Answer: If PORT comes from a method call, it is not a compile-time constant. Reading Config.PORT normally triggers class initialization.

Interview trap: Not every final static field is a compile-time constant.

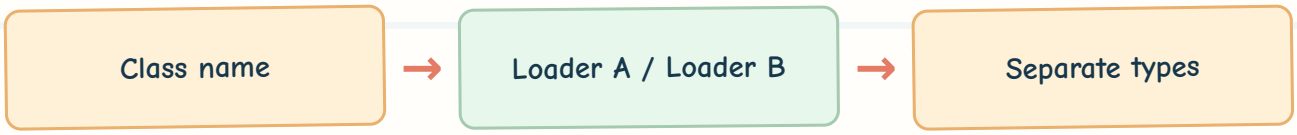
Source guide p. 10 · Technical references J02, J03, J24

Q008. Why can the same class name represent different types?

SHORT ANSWER · SAY THIS FIRST

A Java class is identified by its name AND the class loader that defines it.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A class's identity includes its defining loader as well as its binary name. Two independent defining loaders can produce incompatible types with the same name. This matters for plugins, application servers and redeployment.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Loader A -> example.Plugin
Loader B -> example.Plugin
Same binary name, different defining loader identity
```

Why it matters: A cast can fail even when the displayed type names appear identical.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you diagnose this in a plugin architecture?

Answer: Print each object's getClass().getClassLoader(), inspect duplicated JARs and loader boundaries, then check which API types must be shared.

Interview trap: Looking only at the package and class name can miss a class-loader boundary.

Source guide p. 10 · Technical references J02, J03, J24

Q009 · Class loading, linking and initialization

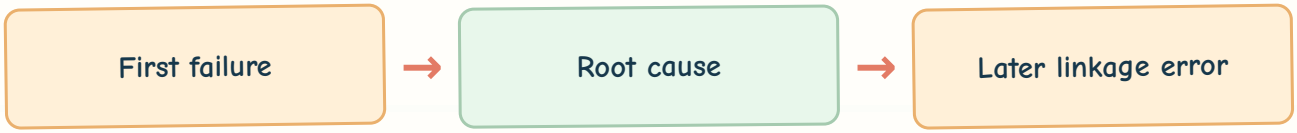
INTERVIEW ANSWER CARD

Q009. How do ClassNotFoundException and NoClassDefFoundError differ?

SHORT ANSWER · SAY THIS FIRST

> ClassNotFoundException often follows an explicit lookup; NoClassDefFoundError is a linkage failure during use.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

ClassNotFoundException commonly reports failure of an explicit class lookup. NoClassDefFoundError is a linkage error when a needed definition cannot be obtained; it can also follow earlier initialization failure. Diagnose the first failure and its cause chain.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Explicit Class.forName -> lookup exception path
Dependent class at execution -> linkage error path
Earlier static-init failure -> later erroneous-class use
```

Why it matters: Classpath visibility and failed initialization are different causes that can produce related symptoms.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why should you preserve the earliest startup exception?

Answer: The first error may show a missing dependency or a failed static initializer; later failures can hide the original reason.

Interview trap: Adding a dependency will not repair a static initializer that throws because configuration is missing.

Source guide p. 10 · Technical references J02, J03, J24

Q010 · Class loading, linking and initialization

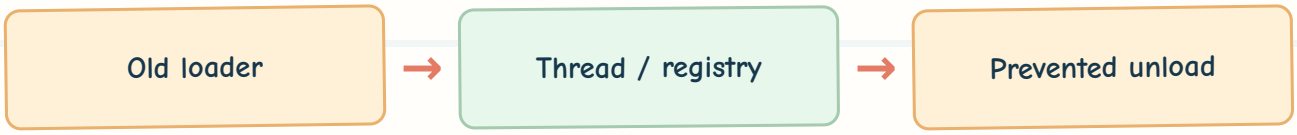
INTERVIEW ANSWER CARD

Q010. How can class loaders leak memory?

SHORT ANSWER · SAY THIS FIRST

> An old class loader stays alive when a long-lived reference still points to its classes, threads or callbacks.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A loader and its classes cannot unload while relevant strong references keep the loader alive. Long-lived threads, ThreadLocals, callbacks or registries can retain application classes across redeployments. Investigate the retaining path rather than only class counts.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Long-lived thread -> context loader -> old deployment
Static registry -> callback -> application class
Repeated reload -> increasing retained metadata
```

Why it matters: The root can belong to infrastructure outside the application being replaced.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you verify that an old deployment loader becomes collectible?

Answer: Stop old deployments, remove references and compare post-GC retained loaders using class-loading logs or heap-dump root paths.

Interview trap: A growing metaspace graph is not proof of a loader leak without lifetime and reachability evidence.

Source guide p. 11 · Technical references J02, J03, J24

CHAPTER 03 / 20

Heap, object layout and native memory

Budget all process memory, not only the heap.



FIVE ORIGINAL INTERVIEW QUESTIONS

011. How are fields and local variables initialized?

012. What is inside a HotSpot object?

013. What are compressed references?

014. What are metaspace and the code cache?

015. How do direct ByteBuffers affect memory?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q011 · Heap, object layout and native memory

INTERVIEW ANSWER CARD

Q011. How are fields and local variables initialized?

SHORT ANSWER · SAY THIS FIRST

> Java assigns defaults to fields, but local variables must be assigned before reading them.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Object and static fields receive default values before their explicit initialization sequence. Local variables must be definitely assigned before use. Reading an unassigned local is a compile-time error rather than automatically reading zero or null.

JAVA / COMMAND / SCENARIO EXAMPLE

```
class Example {
 int count;             // initially 0
 Object value;          // initially null
 void run() { int x; /* printing x is invalid */ }
}
```

Why it matters: Default field values do not imply that every variable gets an implicit source-level initializer.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What values do elements of new int[3] have?

Answer: All three elements are zero because Java initializes new int arrays with the int default value.

Interview trap: Constructor assignment occurs after field storage has its default value.

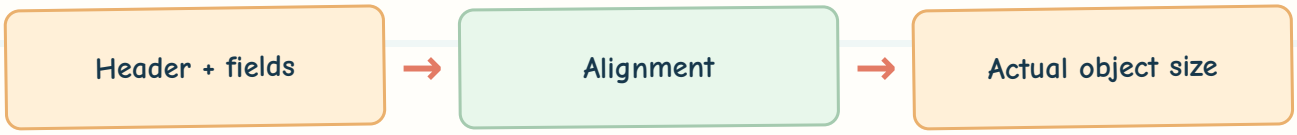
Source guide p. 12 · Technical references J01, J03, J06, J16, J23, J26

Q012. What is inside a HotSpot object?

SHORT ANSWER · SAY THIS FIRST

An object occupies header, fields and possible padding; actual sizes depend on the JVM configuration.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

An object has implementation metadata, instance state and alignment effects. Headers, reference widths and layout depend on architecture, VM options and release. Arrays additionally need length information. Measure layout instead of treating a fixed diagram as a language guarantee.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Measured size = header + fields + padding
References may use compressed representations
Array = metadata + length + elements + padding
```

Why it matters: A collection's retained footprint includes its nodes, arrays and referenced objects.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How can alignment change the effect of adding one field?

Answer: Adding one field may consume spare padding, or push the object to the next alignment boundary. Measure instead of assuming exact bytes.

Interview trap: 'Every object is exactly 16 bytes' is not a reliable statement across layouts.

Source guide p. 13 · Technical references J01, J03, J06, J16, J23, J26

Q013 · Heap, object layout and native memory

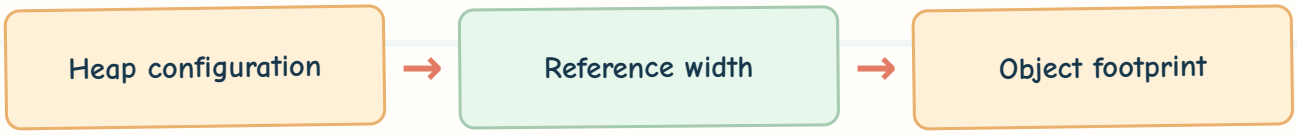
INTERVIEW ANSWER CARD

Q013. What are compressed references?

SHORT ANSWER · SAY THIS FIRST

> Compressed references let compatible HotSpot configurations store many object pointers more compactly.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

HotSpot can encode certain references in a smaller form under applicable heap/layout conditions, decoding them when used. Compressed object pointers and compressed class pointers are distinct mechanisms. Inspect effective flags rather than inferring them only from Xmx.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Inspect effective VM flags on the actual runtime
Compare object layout under selected configurations
Include architecture and object-header mode
```

Why it matters: Smaller references can reduce object footprint and influence cache behavior.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why could changing heap size unexpectedly change memory footprint per object?

Answer: A heap or alignment change can disable/change pointer compression, increasing the size of references inside object graphs.

Interview trap: A memorized heap-size cutoff is not universal across alignment and VM configurations.

Source guide p. 13 · Technical references J01, J03, J06, J16, J23, J26

Q014 · Heap, object layout and native memory

INTERVIEW ANSWER CARD

Q014. What are metaspace and the code cache?

SHORT ANSWER · SAY THIS FIRST

> Metaspace holds class metadata; the code cache holds JIT machine instructions, both outside the Java heap.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

In HotSpot, metaspace holds class-metadata-related native allocations. The code cache holds generated machine code and related structures. These are outside the Java object heap and have different lifecycle and diagnostic signals.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Many class definitions -> metadata growth
Many compiled methods -> code-cache activity
-Xmx -> object heap maximum, not these totals
```

Why it matters: Dynamic proxies, generated classes and application hot paths affect different areas.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which evidence would distinguish metadata exhaustion from a heap leak?

Answer: Use the exact OOME message, class counts/metaspace metrics, heap after GC and native-memory data. Metadata growth with stable live heap suggests a different problem.

Interview trap: Raising Xmx alone does not directly raise a metaspace or code-cache limit.

Source guide p. 13 · Technical references J01, J03, J06, J16, J23, J26

Q015. How do direct ByteBuffers affect memory?

SHORT ANSWER · SAY THIS FIRST

A direct ByteBuffer has a Java wrapper but stores its byte payload in native memory.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A direct ByteBuffer provides a Java object associated with native memory used for its contents. Its cleanup and accounting differ from ordinary heap arrays. Monitor buffer-pool metrics, allocation behavior and process memory; a heap dump alone does not contain all native payloads.

JAVA / COMMAND / SCENARIO EXAMPLE

```
ByteBuffer b = ByteBuffer.allocateDirect(1024);
// Wrapper is a Java object; payload is direct memory.
b.putInt(42);
```

Why it matters: Retained wrappers can keep direct allocations alive.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you diagnose direct-memory exhaustion with a modest Java heap?

Answer: Compare direct-buffer pool use, RSS, NMT where enabled, native allocations and live wrappers. Xmx alone does not explain direct usage.

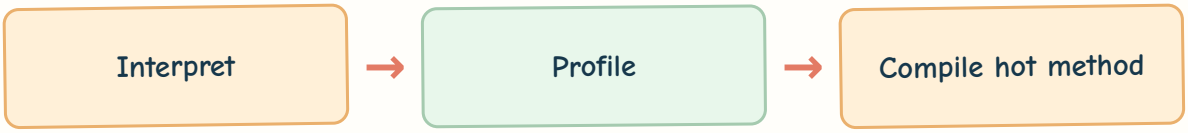
Interview trap: Off-heap allocation is not free memory and does not make the process immune to GC-related lifecycle issues.

Source guide p. 14 · Technical references J01, J03, J06, J16, J23, J26

CHAPTER 04 / 20

JIT compilation and optimization

Warm and cold code can execute differently.



FIVE ORIGINAL INTERVIEW QUESTIONS

016. What is tiered compilation in HotSpot?

017. What is method inlining and why does it matter?

018. What are escape analysis and scalar replacement?

019. What is deoptimization?

020. How do startup and steady-state performance differ?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q016 · JIT compilation and optimization

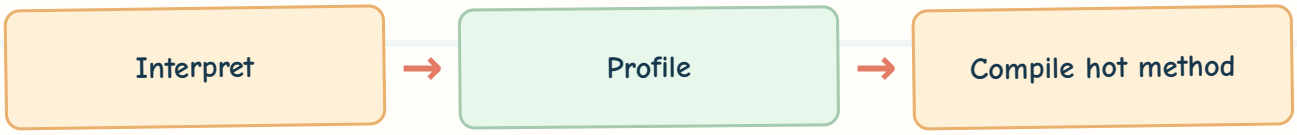
INTERVIEW ANSWER CARD

Q016. What is tiered compilation in HotSpot?

SHORT ANSWER · SAY THIS FIRST

> Tiered compilation combines interpretation, profiling and increasingly optimized native code.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Tiered compilation combines interpretation and different compilation/profiling levels to balance startup and optimized execution. C1 and C2 are common HotSpot compiler names, with actual availability depending on the build. Do not assign a universal invocation threshold to every method.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Interpret -> profile -> compile hot paths
Optimize using observed type and branch behavior
Adapt when assumptions change
```

Why it matters: Early compiled code and later optimized code can have different characteristics.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you determine whether a method was compiled?

Answer: Inspect JIT compilation output where supported, JFR compilation events or diagnostic compiler tooling on the selected HotSpot build.

Interview trap: A fixed '10,000 calls means C2' rule ignores configuration and method behavior.

Source guide p. 15 · Technical references J04-J06, J30-J33

Q017 · JIT compilation and optimization

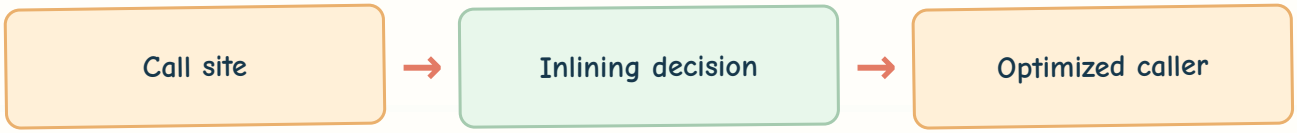
INTERVIEW ANSWER CARD

Q017. What is method inlining and why does it matter?

SHORT ANSWER · SAY THIS FIRST

> Inlining replaces a method call with its body inside compiled code to enable further optimization.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Inlining places a callee's behavior into compiled caller code where profitable. It can remove call overhead and enable other optimizations. Decisions depend on code size, profiles and compiler constraints rather than only a method's source length.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Small hot helper -> potential inline into caller
Inlining -> constants and allocations become visible
Too much expansion -> code size and cache pressure
```

Why it matters: Inlining often matters because it enables optimization across a call boundary.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How might polymorphic call sites affect inlining?

Answer: One type at a call site is easier to optimize; several possible receiver types may need guarded inlining or prevent it.

Interview trap: Making every method final is not a guaranteed performance improvement.

Source guide p. 16 · Technical references J04-J06, J30-J33

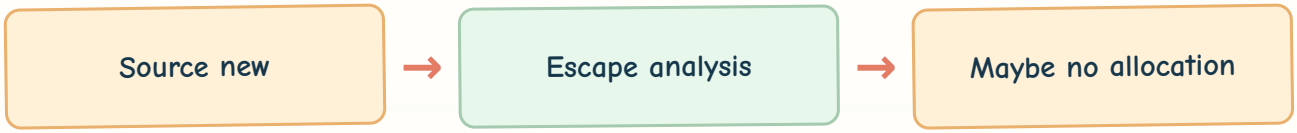
Q018 · JIT compilation and optimization INTERVIEW ANSWER CARD

Q018. What are escape analysis and scalar replacement?

SHORT ANSWER · SAY THIS FIRST

> Escape analysis can prove an object need not be created as a lasting allocation; the JIT may use simple scalar values instead.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Escape analysis reasons about where an object can be observed. Scalar replacement can represent its fields as compiler values and eliminate allocation when permitted. This does not mean that HotSpot universally moves all nonescaping objects onto a physical thread stack.

JAVA / COMMAND / SCENARIO EXAMPLE

```
record Point(int x, int y) {}
int sum() {
 Point p = new Point(2, 3);
 return p.x() + p.y();
}
```

Why it matters: The allocation might disappear in optimized execution without changing observable behavior.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you benchmark an allocation that the compiler can eliminate?

Answer: Use JMH with returned/consumed results and realistic inputs, inspect allocation rates and avoid forcing arbitrary behavior that changes the workload.

Interview trap: A source-level new expression does not prove a heap allocation survives compilation.

Source guide p. 16 · Technical references J04-J06, J30-J33

Q019 · JIT compilation and optimization

INTERVIEW ANSWER CARD

Q019. What is deoptimization?

SHORT ANSWER · SAY THIS FIRST

> Deoptimization safely leaves specialized compiled code when a runtime assumption becomes invalid.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Deoptimization transfers execution from optimized code when assumptions no longer hold or a runtime requirement needs a less specialized representation. It is part of adaptive execution. Frequent transitions can affect performance and should be correlated with profiles and workload changes.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Warm call site sees one implementation
New implementation appears during execution
Compiled assumptions may need adaptation
```

Why it matters: Runtime optimization is speculative but must preserve Java semantics.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What workload change could invalidate a monomorphic type profile?

Answer: Loading a new implementation at a formerly single-type interface call site may invalidate the compiled monomorphic assumption.

Interview trap: A deoptimization event alone is not proof of a defect or the cause of every latency spike.

Source guide p. 16 · Technical references J04-J06, J30-J33

Q020 · JIT compilation and optimization

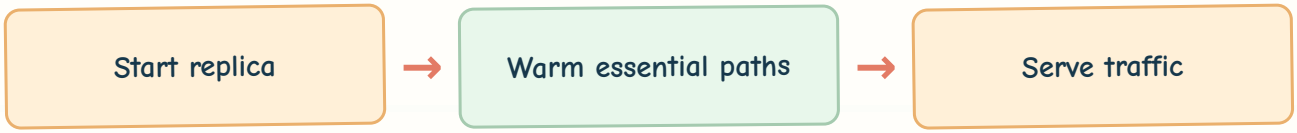
INTERVIEW ANSWER CARD

Q020. How do startup and steady-state performance differ?

SHORT ANSWER · SAY THIS FIRST

> Startup measures time until useful service; steady-state measures performance after warmup at a defined load.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Startup includes loading, initialization, dependency creation and compilation activity. Steady-state measurements assume a defined warmed and representative workload. Optimize each objective separately and include deployment/resource constraints.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Startup objective: ready for traffic in bounded time
Steady state: latency and throughput at target load
Recovery: new replicas become useful quickly enough
```

Why it matters: A change that improves warm throughput can increase startup cost or memory.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would rapid autoscaling change your warmup strategy?

Answer: Warm capacity or pre-warming may be needed. Measure how quickly fresh replicas pass readiness and meet real p95/p99 under traffic.

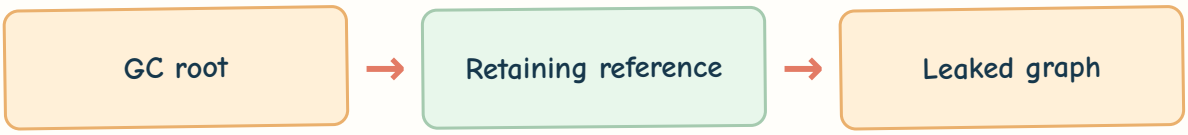
Interview trap: Measuring only the first request misrepresents long-running service capacity.

Source guide p. 17 · Technical references J04-J06, J30-J33

CHAPTER 05 / 20

Garbage collection fundamentals

Reachability, not source-code scope, defines liveness.



FIVE ORIGINAL INTERVIEW QUESTIONS

021. How does the GC decide an object is garbage?

022. What are GC roots?

023. What are young and old generations?

024. How do marking, sweeping and compaction differ?

025. What is the difference between parallel, concurrent and stop-the-world GC?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

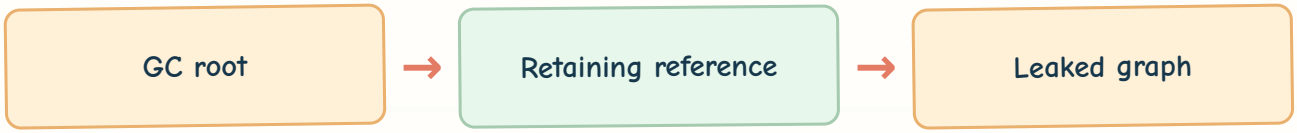
Q021 · Garbage collection fundamentals INTERVIEW ANSWER CARD

Q021. How does the GC decide an object is garbage?

SHORT ANSWER · SAY THIS FIRST

> GC finds reachable objects from roots; unreachable objects can be collected later.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Tracing collectors discover objects reachable from relevant roots and references under their processing rules. Objects no longer reachable can be reclaimed. Reachability is a runtime property and reclamation timing is not a deterministic source-code event.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Root -> service -> cache -> retained value
No remaining retaining path -> eligible for reclamation
GC cycle -> may recover storage
```

Why it matters: An object can be logically useless to the business while remaining strongly reachable.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why can Java still have memory leaks?

Answer: Java leaks occur when code still retains objects it no longer needs, such as growing caches, listeners or ThreadLocal values.

Interview trap: A reference going out of lexical scope does not guarantee immediate collection.

Source guide p. 18 · Technical references J07-J09, J25

Q022 · Garbage collection fundamentals

INTERVIEW ANSWER CARD

Q022. What are GC roots?

SHORT ANSWER · SAY THIS FIRST

GC roots are live starting points such as thread state, runtime references and JNI handles.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Roots are starting points used to discover reachable objects, such as relevant thread state, runtime-maintained references and JNI handles. Exact implementation categories vary. Root-path analysis explains why a supposedly unused object remains alive.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Thread state -> local reference -> object graph
Runtime class/static state -> long-lived registry
JNI handle -> Java object
```

Why it matters: The retaining chain can be more informative than the object's own size.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which root might retain a request through a thread pool?

Answer: A pooled worker's ThreadLocal value or a queued task can retain request context between requests.

Interview trap: Treating all static fields as permanent immortal storage ignores class-loader lifetime.

Source guide p. 19 · Technical references J07-J09, J25

Q023 · Garbage collection fundamentals

INTERVIEW ANSWER CARD

Q023. What are young and old generations?

SHORT ANSWER · SAY THIS FIRST

Young/old treatment helps collectors optimize for short-lived versus long-lived objects.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Generational collectors exploit the observation that many allocated objects become unreachable quickly. Young collection focuses on recently allocated objects; survivors can remain young or move to older treatment according to policy. Generations and layouts differ by collector and release.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Temporary JSON objects -> often short lived
Cached customer graph -> may remain live
Promotion policy -> implementation decision
```

Why it matters: Object lifetime distribution affects collection cost and survivor volume.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why does a sudden rise in survivors matter more than allocation count alone?

Answer: More survivors mean more objects must be copied or retained, increasing evacuation work and old-generation pressure.

Interview trap: Every collector and JDK does not use the same Eden/two-survivor contiguous layout.

Source guide p. 19 · Technical references J07-J09, J25

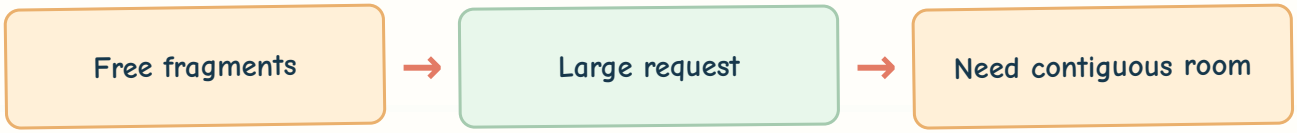
Q024 · Garbage collection fundamentals INTERVIEW ANSWER CARD

Q024. How do marking, sweeping and compaction differ?

SHORT ANSWER · SAY THIS FIRST

> Mark finds live objects, sweep reclaims dead space, and compaction moves live objects together.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Marking identifies live objects. Sweeping makes dead storage reusable. Compaction relocates live objects to reduce fragmentation; copying/evacuation also relocates survivors into other storage. Collectors combine these techniques in different phases.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Mark: determine live graph
Sweep: reclaim dead locations
Compact/evacuate: relocate live objects
Update references: preserve valid access
```

Why it matters: Free bytes and a usable contiguous allocation are different properties.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why can a large allocation fail despite some free heap space?

Answer: Free bytes can be scattered into small gaps; a large contiguous allocation may need compaction or fail.

Interview trap: A concurrent collector is not necessarily a nonmoving collector.

Source guide p. 19 · Technical references J07-J09, J25

Q025 · Garbage collection fundamentals

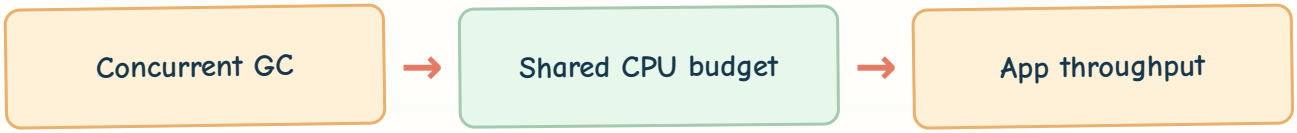
INTERVIEW ANSWER CARD

Q025. What is the difference between parallel, concurrent and stop-the-world GC?

SHORT ANSWER · SAY THIS FIRST

Parallel means several GC workers; concurrent means GC overlaps application execution; stop-the-world means app threads pause.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Parallel describes multiple GC workers doing work together. Concurrent describes GC work overlapping application execution. Stop-the-world describes phases that suspend participating application execution. A collector can use all three at different times.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Parallel STW phase: many workers, app paused
Concurrent phase: GC and app both consume CPU
Pause target: objective, not an automatic guarantee
```

Why it matters: Concurrency shifts some work out of pauses but still consumes resources and requires coordination.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why can a low-pause collector still reduce application throughput?

Answer: Concurrent GC uses CPU and memory bandwidth while the app runs, so throughput may fall despite shorter pauses.

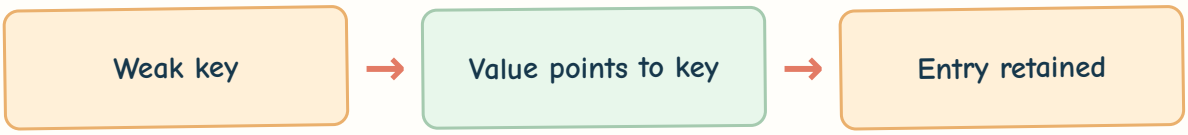
Interview trap: 'Concurrent means no pauses' is inaccurate.

Source guide p. 20 · Technical references J07-J09, J25

CHAPTER 06 / 20

References, cleanup and retention leaks

Remove the retaining owner and close scarce resources.



FIVE ORIGINAL INTERVIEW QUESTIONS

026. How do strong, soft, weak and phantom references differ?

027. Why should external resources use try-with-resources?

028. Why is finalization unsuitable for normal cleanup?

029. How do ThreadLocals leak or cross-contaminate requests?

030. How do you distinguish an intentional cache from a memory leak?

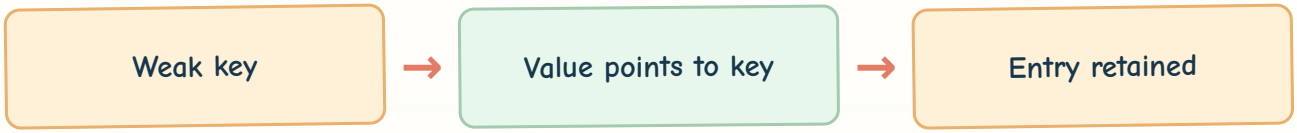
> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q026. How do strong, soft, weak and phantom references differ?

SHORT ANSWER · SAY THIS FIRST

Strong, soft, weak and phantom references offer different reachability and cleanup behaviors.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Ordinary strong reachability prevents reclamation. Soft references support memory-sensitive retention with collector-controlled clearing. Weak references do not preserve weakly reachable referents. Phantom references support post-reachability notification patterns and do not return a referent through get.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Strong: ordinary object field
Soft: memory-sensitive retention, no fixed cache SLA
Weak: association that should not keep referent alive
Phantom: lifecycle notification through a queue
```

Why it matters: Reference queues can inform cleanup bookkeeping without promising an exact time.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why does WeakHashMap still require careful value design?

Answer: A WeakHashMap value can strongly refer back to its key, making the key reachable and preventing the expected removal.

Interview trap: SoftReference is not a predictable bounded cache replacement policy.

Source guide p. 21 · Technical references J17, J24-J27

Q027 · References, cleanup and retention leaks

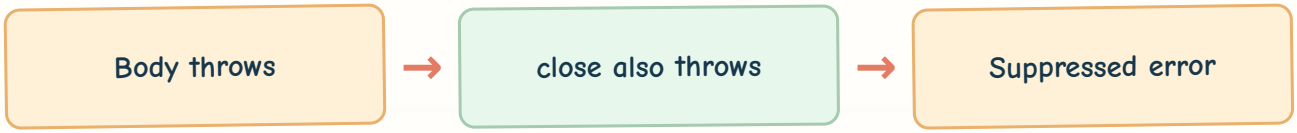
INTERVIEW ANSWER CARD

Q027. Why should external resources use try-with-resources?

SHORT ANSWER · SAY THIS FIRST

> Use try-with-resources to close sockets, files and DB handles promptly even when Java memory is fine.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Files, sockets and database connections have limited external resources and need timely release. AutoCloseable lets structured code release them deterministically on scope exit. GC reclaims Java memory on its own schedule and is not a resource-management policy.

JAVA / COMMAND / SCENARIO EXAMPLE

```
try (var in = java.nio.file.Files.newInputStream(path)) {
 // read required data
}
```

Why it matters: Close runs even when the body throws, subject to normal abrupt termination constraints.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What happens when both the body and close throw?

Answer: The body exception remains primary; close failures are recorded as suppressed exceptions accessible with getSuppressed().

> **Interview trap:** Collecting an object is not a guarantee that its external resource was released promptly.

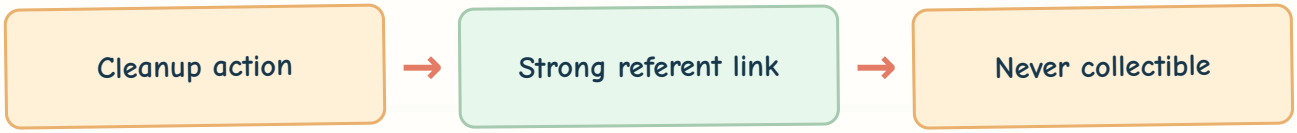
Source guide p. 22 · Technical references J17, J24-J27

Q028. Why is finalization unsuitable for normal cleanup?

SHORT ANSWER · SAY THIS FIRST

Do not depend on finalizers to close resources; their execution is uncertain and they are deprecated for removal.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Finalization has unpredictable timing and problematic reliability and security properties; it is deprecated for removal in modern Java. Prefer explicit close. Cleaner can provide a carefully designed fallback for suitable resources but must not capture the referent and prevent reclamation.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Primary lifecycle: explicit close()
Fallback where appropriate: independent cleanup state
Avoid: relying on finalize() before resource exhaustion
```

Why it matters: Cleanup ownership should be visible in normal control flow.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How can a cleanup action accidentally retain the object it should clean?

Answer: A Cleaner action that holds the referent directly creates a strong path to it; keep independent cleanup state instead.

Interview trap: Cleaner is not a deterministic substitute for close.

Source guide p. 22 · Technical references J17, J24-J27

Q029 · References, cleanup and retention leaks

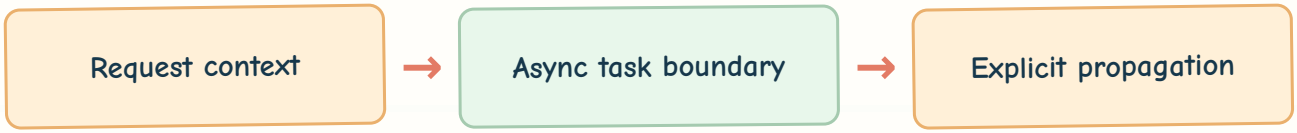
INTERVIEW ANSWER CARD

Q029. How do ThreadLocals leak or cross-contaminate requests?

SHORT ANSWER · SAY THIS FIRST

Pooled threads outlive requests, so an uncleared ThreadLocal may leak data or expose it to the next request.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Values stored on long-lived threads can survive a request unless removed or replaced. ThreadLocal maps have implementation lifecycle details that do not guarantee prompt value release. Use finally to clean scoped state and avoid carrying one user's context into another task.

JAVA / COMMAND / SCENARIO EXAMPLE

```
try {
 context.set(requestContext);
 handleRequest();
} finally {
 context.remove();
}
```

Why it matters: Pool threads outlive the request that created the value.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would asynchronous work complicate context propagation?

Answer: Tasks can move between threads, so ThreadLocal context does not reliably follow them. Pass context explicitly or use a deliberately scoped mechanism.

Interview trap: A weak reference to a ThreadLocal key does not guarantee its associated value immediately disappears.

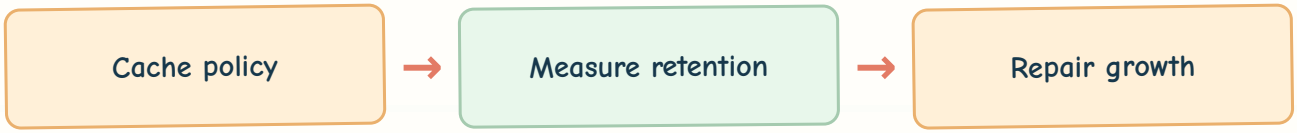
Source guide p. 23 · Technical references J17, J24-J27

Q030. How do you distinguish an intentional cache from a memory leak?

SHORT ANSWER · SAY THIS FIRST

A cache is intentional only when it has a useful size/TTL policy and a measurable eviction plan.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Compare retention with the defined policy and workload. A cache that exceeds its intended size/TTL or retains unnecessary object graphs is a correctness or design problem. Track post-reclamation live-set trends and inspect retaining paths before labeling every rise a leak.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Cache policy: bounded entries and measured byte budget
Observe: retained graph after comparable workload phases
Investigate: owners, eviction and references
```

Why it matters: A flat object count can still hide larger values.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which cache metrics would you record alongside a heap comparison?

Answer: Track entries, estimated bytes, hit/miss rate, evictions, load failures, live-set size and the cache's effect on backend calls.

Interview trap: A single heap snapshot cannot prove a growth trend.

Source guide p. 23 · Technical references J17, J24-J27

CHAPTER 07 / 20

Choosing a garbage collector

Select a collector for a measured service objective.



FIVE ORIGINAL INTERVIEW QUESTIONS

031. How do you choose a collector?

032. When is Serial GC useful?

033. When is Parallel GC useful?

034. What is Epsilon GC and why is it unusual?

035. What happened to CMS and what is the modern default?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q031 · Choosing a garbage collector

INTERVIEW ANSWER CARD

Q031. How do you choose a collector?

SHORT ANSWER · SAY THIS FIRST

> Choose a collector by measuring user latency, throughput, CPU and memory on the real workload.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Define acceptable tail latency, throughput, memory footprint and CPU budget, then test suitable collectors on the same runtime and representative workload. Start with supported defaults. Verify collector availability in the actual vendor build and platform.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Compare: same app, heap, load and resource limits
Measure: throughput, p99, GC pauses, CPU and RSS
Keep: correctness, error rate and startup observations
```

Why it matters: Collector selection is an experiment constrained by service objectives.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What evidence would justify changing from G1 to ZGC?

Answer: Move from G1 to ZGC only if a repeatable improvement in tail latency outweighs CPU, heap/RSS and throughput costs under comparable load.

Interview trap: Smaller GC pauses alone do not establish a better application.

Source guide p. 24 · Technical references J08, J18, J20, J36, J37

Q032 · Choosing a garbage collector

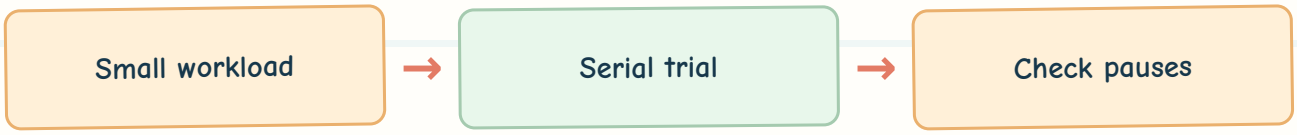
INTERVIEW ANSWER CARD

Q032. When is Serial GC useful?

SHORT ANSWER · SAY THIS FIRST

> Serial GC uses one GC worker; it may suit tiny heaps or tightly limited CPUs when pauses are acceptable.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Serial GC performs collection with a single GC worker and is suited to some small-footprint or limited-resource workloads. Evaluate its simplicity and overhead against pause behavior for the actual live set. It is not a generic choice for all latency-sensitive services.

JAVA / COMMAND / SCENARIO EXAMPLE

```
java -XX:+UseSerialGC -Xmx128m SmallApp
```

Why it matters: A small workload may not benefit from a large parallel collection workforce.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would you measure before using Serial in a small container?

Answer: Measure p95/p99 pauses, throughput, peak RSS, warmup, CPU quota and total job duration with realistic data.

> **Interview trap:** A low thread count does not guarantee acceptable pauses as the live set grows.

Source guide p. 25 · Technical references J08, J18, J20, J36, J37

Q033 · Choosing a garbage collector INTERVIEW ANSWER CARD

Q033. When is Parallel GC useful?

SHORT ANSWER · SAY THIS FIRST

> Parallel GC prioritizes useful throughput while allowing stop-the-world collection work.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Parallel GC emphasizes throughput using multiple workers for stop-the-world collection. It can fit batch workloads with acceptable pause tolerance. Compare total completion time and resource use rather than assuming concurrent collection is always superior.

JAVA / COMMAND / SCENARIO EXAMPLE

```
java -XX:+UseParallelGC -Xmx2g BatchJob
```

Why it matters: More parallel collection work can reduce some collection durations when CPU is available.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why might a batch job prefer Parallel over a lower-pause collector?

Answer: For a batch job with flexible pauses, faster completion and lower total CPU/memory cost may matter more than p99 response latency.

Interview trap: Parallel workers compete within limited CPU quotas and can still cause substantial application pauses.

Source guide p. 25 · Technical references J08, J18, J20, J36, J37

Q034 · Choosing a garbage collector

INTERVIEW ANSWER CARD

Q034. What is Epsilon GC and why is it unusual?

SHORT ANSWER · SAY THIS FIRST

Epsilon allocates but never reclaims objects; it is useful only for bounded experiments.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Epsilon manages allocation without reclaiming unused objects. It is useful for bounded experiments and specialized workloads that complete within their memory budget. In HotSpot it requires experimental-option unlocking and is not an ordinary long-running service collector.

JAVA / COMMAND / SCENARIO EXAMPLE

```
java -XX:+UnlockExperimentalVMOptions \
  -XX:+UseEpsilonGC -Xmx128m BoundedExperiment
```

Why it matters: Once available heap is exhausted, unreachable objects are not recovered for reuse by Epsilon.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you bound a useful Epsilon experiment?

Answer: Choose a fixed operation count, -Xmx, input size and duration; run in an isolated process and expect exhaustion once the budget is spent.

Interview trap: 'No GC pause' is not proof of a sustainable memory lifecycle.

Source guide p. 25 · Technical references J08, J18, J20, J36, J37

Q035 · Choosing a garbage collector

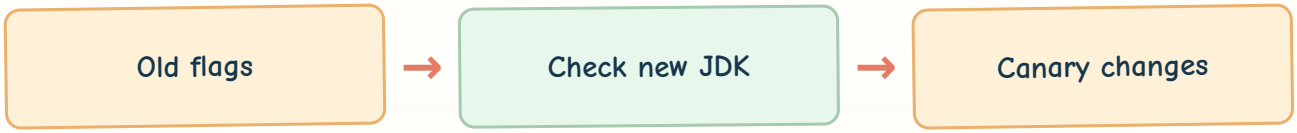
INTERVIEW ANSWER CARD

Q035. What happened to CMS and what is the modern default?

SHORT ANSWER · SAY THIS FIRST

> CMS was removed in JDK 14; G1 is a common server default, but always inspect runtime flags.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

CMS was removed in JDK 14. G1 became the default for common server configurations in JDK 9, with ergonomic and build/platform qualifications. Inspect the effective collector instead of copying a historical CMS flag set into a modern launch command.

JAVA / COMMAND / SCENARIO EXAMPLE

```
java -XX:+PrintCommandLineFlags -version
# Confirm the collector selected by this runtime.
```

Why it matters: Historical guides can contain options that newer JDKs reject.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What is your migration process for old production JVM flags?

Answer: Inventory all launch flags, confirm target-JDK support, remove obsolete settings, test performance and failure cases, then canary the rollout.

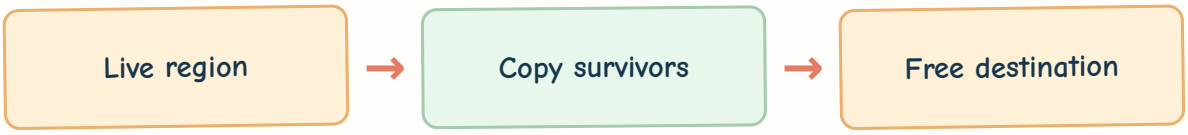
Interview trap: Do not describe CMS as an available standard collector in Java 17, 21 or 25.

Source guide p. 26 · Technical references J08, J18, J20, J36, J37

CHAPTER 08 / 20

G1 architecture and phases

Understand regions, evacuation and old-space pressure.



FIVE ORIGINAL INTERVIEW QUESTIONS

036. How does G1 organize the heap?

037. What is a G1 young collection?

038. What is a G1 mixed collection?

039. What do remembered sets and barriers do in G1?

040. What are humongous objects in G1?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q036 · G1 architecture and phases

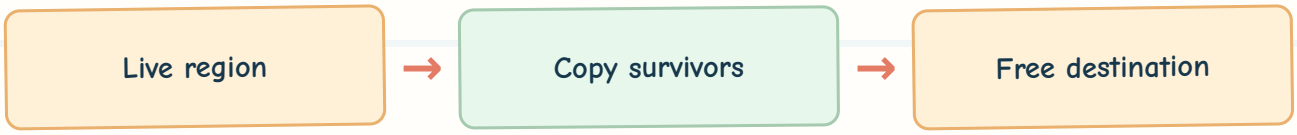
INTERVIEW ANSWER CARD

Q036. How does G1 organize the heap?

SHORT ANSWER · SAY THIS FIRST

> G1 divides the heap into regions that can play Eden, survivor, old or humongous roles.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

G1 divides its heap into equally sized regions used for young, old, free and humongous allocation roles. The generational arrangement is logical rather than one permanently contiguous young block. Region selection helps spread reclamation work across pauses.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Regions: Eden | Survivor | Old | Free | Humongous
Collection set: selected regions for this evacuation
Live objects: copied to suitable destination space
```

Why it matters: Region roles can change as storage is reused.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why does evacuation require destination capacity?

Answer: Live objects need free destination regions when copied; without enough suitable space, evacuation can fail.

Interview trap: A region map is not a fixed Java object-addressing contract.

Source guide p. 27 · Technical references J09, J10

Q037 · G1 architecture and phases

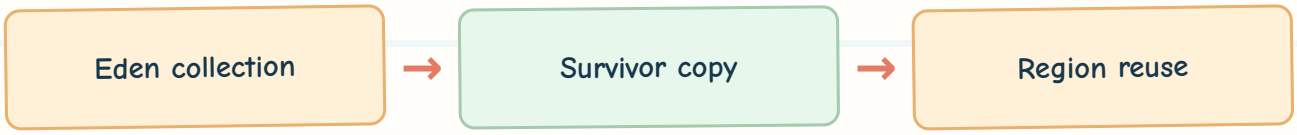
INTERVIEW ANSWER CARD

Q037. What is a G1 young collection?

SHORT ANSWER · SAY THIS FIRST

A G1 young GC copies reachable objects from selected young regions and reclaims the rest.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A young evacuation pause processes the young collection set, relocating live survivors and making the collected regions reusable. Survival, root work and reference processing affect duration. Frequent young pauses can be normal for an allocation-heavy service.

JAVA / COMMAND / SCENARIO EXAMPLE

```
New allocations -> Eden regions
Young pause -> survivors copied
Dead temporary objects -> storage reused
```

Why it matters: Allocation frequency and surviving volume are separate tuning inputs.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why might fewer allocations still produce a slower pause?

Answer: Pause time also depends on survivor bytes, references, remembered-set/root work and CPU scheduling, not just allocation count.

Interview trap: Every young pause is not evidence of a leak.

Source guide p. 28 · Technical references J09, J10

Q038 · G1 architecture and phases

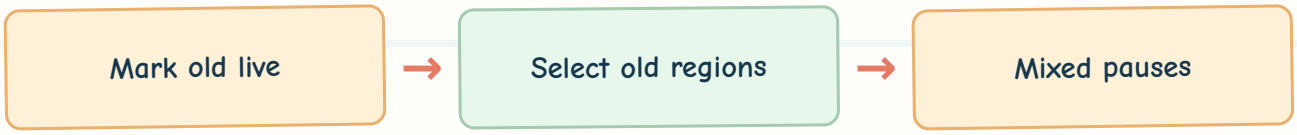
INTERVIEW ANSWER CARD

Q038. What is a G1 mixed collection?

SHORT ANSWER · SAY THIS FIRST

A G1 mixed collection includes young regions plus selected older regions after marking.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A mixed evacuation pause collects young regions plus selected old regions after marking has identified suitable candidates. It differs from collecting the whole heap. Observe which portions of evacuation work contribute to the pause.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Marking -> old-region liveness information
Mixed collection set -> young + selected old regions
Sequence of pauses -> incremental old reclamation
```

Why it matters: Old-space work is distributed rather than necessarily reclaimed all at once.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What tradeoff follows from spreading old reclamation across more pauses?

Answer: More smaller pauses may reduce individual duration but increase frequency and overhead. Compare both tail latency and useful throughput.

Interview trap: 'Mixed GC' is not synonymous with 'Full GC'.

Source guide p. 28 · Technical references J09, J10

Q039 · G1 architecture and phases

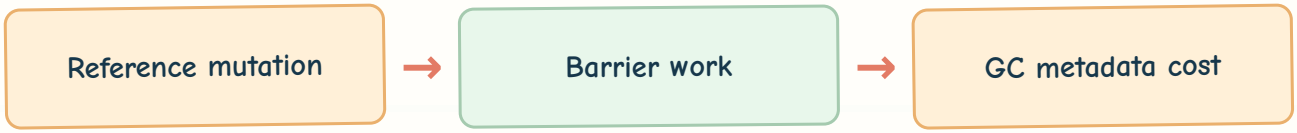
INTERVIEW ANSWER CARD

Q039. What do remembered sets and barriers do in G1?

SHORT ANSWER · SAY THIS FIRST

> G1 remembered sets track relevant references across regions so a young collection need not scan the entire heap.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

They help track relevant cross-region references so selected regions can be collected without blindly rescanning the entire heap every time. Barriers record information when references change; maintaining and processing it costs CPU and memory.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Old region object -> reference into selected young region
Recorded cross-region information -> root discovery
Evacuation -> update relevant references
```

Why it matters: The application reference pattern influences collection overhead.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why might heavy reference mutation affect GC cost?

Answer: Frequent pointer writes can increase write-barrier and remembered-set update work, which costs CPU and memory.

> **Interview trap:** Remembered sets do not contain every object as a complete backup copy.

Source guide p. 28 · Technical references J09, J10

Q040 · G1 architecture and phases

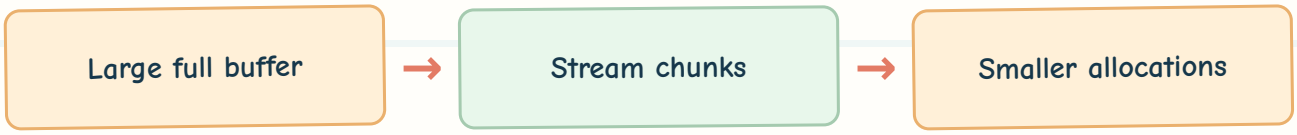
INTERVIEW ANSWER CARD

Q040. What are humongous objects in G1?

SHORT ANSWER · SAY THIS FIRST

Very large G1 objects use special humongous regions, which can increase space and fragmentation pressure.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Objects at or above roughly half a G1 region, including the threshold defined by the implementation, receive special allocation treatment across contiguous regions. This can increase old-space pressure and fragmentation. Inspect actual region size and humongous allocation evidence.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Example region: 1 MiB
Large byte array: inspect total aligned object size
Payload buffering -> possible humongous allocation
```

Why it matters: Large arrays used for request bodies can change the collector's allocation path.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would streaming a payload alter this behavior?

Answer: Stream bounded chunks instead of building one giant byte array; then check humongous allocation events, p99 and memory peaks.

Interview trap: The exact boundary is based on object size and the selected region size, not a universal payload byte count.

Source guide p. 29 · Technical references J09, J10

CHAPTER 09 / 20

Evidence-driven G1 tuning

Change a flag only after measuring a phase.



FIVE ORIGINAL INTERVIEW QUESTIONS

041. Does MaxGCPauseMillis guarantee a maximum pause?

042. Why should you avoid fixing young-generation size blindly for G1?

043. What causes G1 evacuation failure or Full GC?

044. How does initiating heap occupancy affect G1?

045. How should you respond to long mixed pauses or humongous pressure?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q041 · Evidence-driven G1 tuning

INTERVIEW ANSWER CARD

Q041. Does MaxGCPauseMillis guarantee a maximum pause?

SHORT ANSWER · SAY THIS FIRST

MaxGCPauseMillis is a G1 planning target, not a maximum-pause promise.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

No. For G1 it is a soft target used in collection planning, not a hard real-time bound. A tighter target can change collection frequency and overhead without guaranteeing application p99. Keep the target tied to a measured service objective.

JAVA / COMMAND / SCENARIO EXAMPLE

```
-XX:MaxGCPauseMillis=100
# Illustrative target; not a 100 ms guarantee.
```

Why it matters: Target planning operates within live-set, capacity and CPU constraints.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What should you measure after lowering the target?

Answer: Measure actual pause percentiles, frequency, GC CPU, throughput, allocation stalls and the service's end-to-end p99.

Interview trap: A target lower than the available work budget cannot make necessary work vanish.

Source guide p. 30 · Technical references J09, J10

Q042 · Evidence-driven G1 tuning

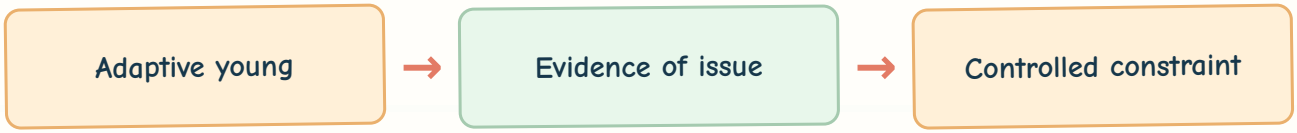
INTERVIEW ANSWER CARD

Q042. Why should you avoid fixing young-generation size blindly for G1?

SHORT ANSWER · SAY THIS FIRST

G1 sizes young collections adaptively; forcing a fixed young size may prevent useful adaptation.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

G1 adapts its young collection set to meet objectives. Options that constrain young sizing can reduce this freedom and create unexpected behavior. First inspect allocation, survival and pause phases; change constraints only for a specific measured hypothesis.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Baseline: selected G1 defaults and measured heap
Avoid inherited -Xmn without understanding its effect
Compare pause phases and useful throughput
```

Why it matters: A flag that helped another collector may interfere with G1's policy.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What evidence would justify constraining G1's young generation?

Answer: Only constrain it if logs show a repeatable phase-specific problem and controlled A/B testing improves user latency without harming throughput.

Interview trap: There is no universal 'young must be 30% of heap' rule.

Source guide p. 30 · Technical references J09, J10

Q043 · Evidence-driven G1 tuning

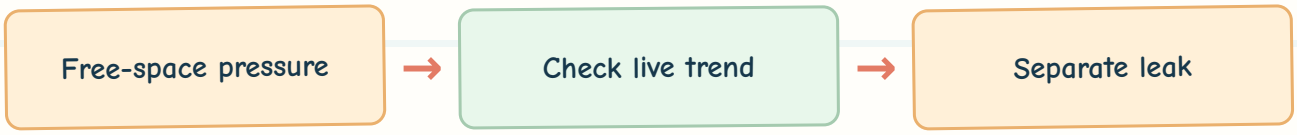
INTERVIEW ANSWER CARD

Q043. What causes G1 evacuation failure or Full GC?

SHORT ANSWER · SAY THIS FIRST

> G1 may fail evacuation when too much live data must move but too little suitable free space remains.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Insufficient suitable destination space, high retained occupancy, late reclamation and allocation patterns can prevent successful evacuation. Full collection can follow, but explicit requests or other causes also exist. Read the stated cause and preceding events.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Inspect: evacuation-failure evidence
Compare: live set, free regions and allocation spikes
Correlate: marking progress and CPU availability
```

Why it matters: The first relevant warning often explains the later long pause.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you distinguish capacity pressure from a leaking cache?

Answer: Check whether post-GC live set keeps rising under equal workloads and inspect cache ownership/eviction; a stable live set with sharp peaks suggests capacity pressure.

Interview trap: Raising heap without addressing unbounded retention may only postpone recurrence.

Source guide p. 30 · Technical references J09, J10

Q044 · Evidence-driven G1 tuning

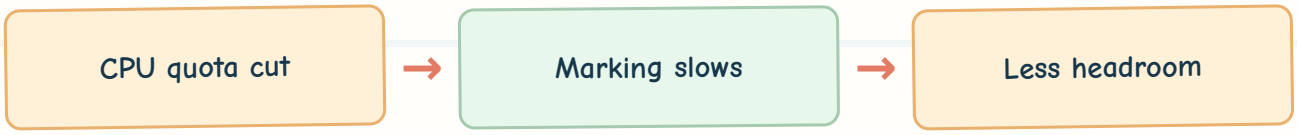
INTERVIEW ANSWER CARD

Q044. How does initiating heap occupancy affect G1?

SHORT ANSWER · SAY THIS FIRST

> G1 starts concurrent marking based on occupancy and predictions; it needs CPU time to finish.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

G1 uses occupancy and adaptive predictions to decide when to begin marking. Earlier marking can create time to reclaim old regions, while CPU constraints can still prevent timely completion. Inspect adaptive behavior before forcing a static threshold.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Observe marking start versus old occupancy
Check allocation during the marking cycle
Verify CPU headroom for concurrent work
```

Why it matters: The useful variable is time available before pressure, not one threshold in isolation.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would happen if a container's CPU limit is cut sharply?

Answer: Less CPU can delay concurrent marking, lengthen wall-clock work and increase risk of old-space pressure or Full GC.

Interview trap: A copied IHOP value is not a guarantee that marking finishes before exhaustion.

Source guide p. 31 · Technical references J09, J10

Q045 · Evidence-driven G1 tuning

INTERVIEW ANSWER CARD

Q045. How should you respond to long mixed pauses or humongous pressure?

SHORT ANSWER · SAY THIS FIRST

> Diagnose which G1 work is slow before tuning; large buffers or retention may be better fixed in code.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Use phase and heap evidence to separate old evacuation, object copying, roots and fragmentation. Reduce problematic allocation/retention where possible. Collector-specific flags are a controlled alternative whose throughput and memory effects must be tested.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Hypothesis: large response buffering creates pressure
Change: stream bounded chunks
Validate: fewer large allocations, stable throughput
Check: p99, heap trend, CPU and errors
```

Why it matters: An application change can remove the cause instead of shifting collector overhead.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you isolate the effect of a region-size experiment?

Answer: Hold JDK, heap, inputs, traffic and all other flags constant; change only region size, repeat runs and compare large-object counts, pauses and throughput.

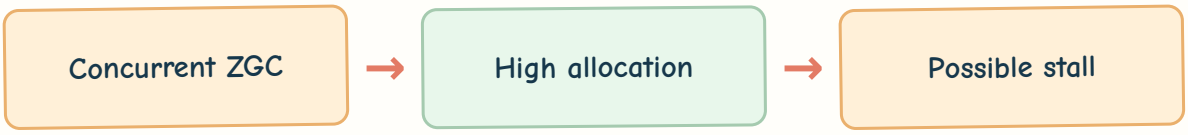
Interview trap: Changing region size affects more than one metric and is not a universal fragmentation cure.

Source guide p. 31 · Technical references J09, J10

CHAPTER 10 / 20

ZGC and Shenandoah

Low pause work still needs memory and CPU headroom.



FIVE ORIGINAL INTERVIEW QUESTIONS

046. What makes ZGC different from G1?

047. How did ZGC change across Java 17, 21 and 25?

048. How do you size heap for a concurrent collector?

049. What is Shenandoah and is it available everywhere?

050. What changed for Generational Shenandoah in Java 25?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q046 · ZGC and Shenandoah

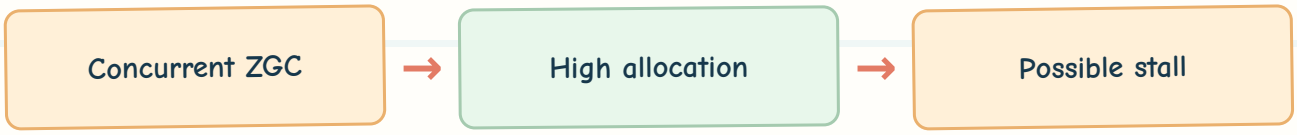
INTERVIEW ANSWER CARD

Q046. What makes ZGC different from G1?

SHORT ANSWER · SAY THIS FIRST

> ZGC moves more GC work into concurrent phases to keep pauses low; G1 evacuates selected regions during pauses.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

ZGC emphasizes low pauses with concurrent work, including relocation, using its implementation-specific reference/barrier mechanisms. G1 evacuates selected regions during pauses and balances throughput and latency. Compare full application results under the same resources.

JAVA / COMMAND / SCENARIO EXAMPLE

```
G1: pause-based evacuation of selected regions
ZGC: substantial concurrent marking and relocation
Both: require memory and CPU budgets
```

Why it matters: Moving work out of pauses does not remove its resource cost.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What might cause allocation stalls despite short ZGC pauses?

Answer: If allocation exceeds reclamation progress, little heap headroom or CPU starvation can cause an allocation stall despite tiny STW pauses.

Interview trap: A low-pause collector cannot guarantee low end-to-end request latency.

Source guide p. 32 · Technical references J11-J15, J40

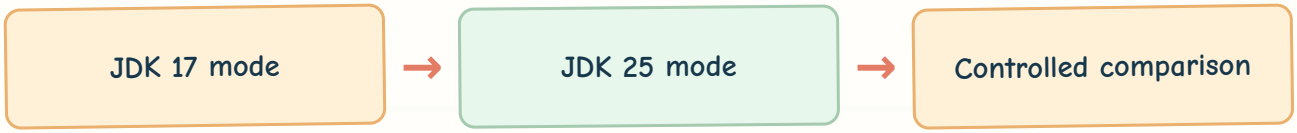
Q047 · ZGC and Shenandoah INTERVIEW ANSWER CARD

Q047. How did ZGC change across Java 17, 21 and 25?

SHORT ANSWER · SAY THIS FIRST

> Java 17 ZGC is non-generational; Java 21 can opt into generational ZGC; Java 25 uses generational ZGC.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Java 17 ZGC is non-generational. Java 21 introduced an opt-in generational mode. Generational mode became the default for ZGC in Java 23; Java 24 removed the non-generational mode. Java 25 uses generational ZGC without the old mode-selection option.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Java 17: -XX:+UseZGC
Java 21: -XX:+UseZGC -XX:+ZGenerational
Java 25: -XX:+UseZGC
```

Why it matters: The mode is selected within ZGC; this does not make ZGC the general default collector.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What must you compare when moving a Java 17 ZGC workload to Java 25?

Answer: Compare collector mode, effective flags, heap/RSS, CPU, allocation stalls, pause distribution, throughput, startup and error rates on the real service.

Interview trap: Do not carry ZGenerational into a Java 25 launch recipe.

Source guide p. 33 · Technical references J11-J15, J40

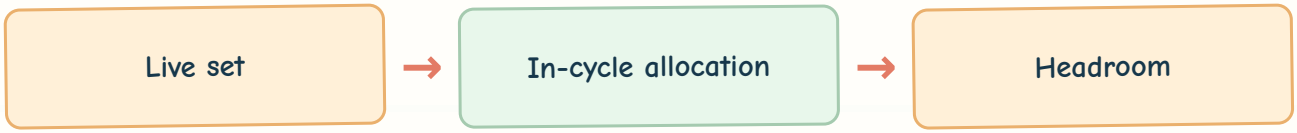
Q048 · ZGC and Shenandoah INTERVIEW ANSWER CARD

Q048. How do you size heap for a concurrent collector?

SHORT ANSWER · SAY THIS FIRST

> Concurrent collectors need space for the live graph plus objects allocated while GC is running.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Provide room for the retained live set and allocations that occur while reclamation proceeds. CPU headroom and allocation spikes determine whether concurrent work keeps up. Use observed stalls and memory behavior instead of assigning an arbitrary percentage to every service.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Heap need: live set + in-cycle allocation + headroom
CPU constraint: collector must make progress
Observe: allocation stalls and useful throughput
```

Why it matters: A heap close to the live set can fail to provide adequate concurrent breathing room.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you tune for bursts with a stable long-term live set?

Answer: Measure short burst allocation rates and cycle duration; add observed safety headroom and check worst-case stalls within the memory limit.

Interview trap: 'Only the live set matters' overlooks ongoing allocations.

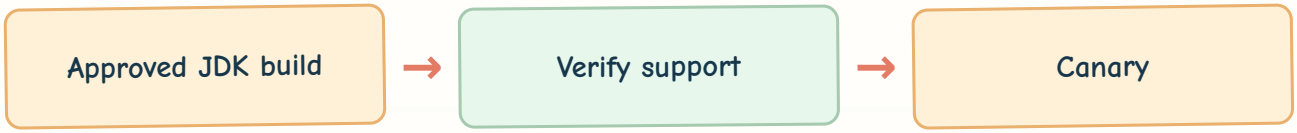
Source guide p. 33 · Technical references J11-J15, J40

Q049. What is Shenandoah and is it available everywhere?

SHORT ANSWER · SAY THIS FIRST

> Shenandoah aims for low pauses with concurrent work, but it is not available in every JDK build.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Shenandoah is a low-pause HotSpot collector using concurrent work, including evacuation. Availability depends on JDK distribution, build and platform. Verify the runtime directly; Oracle and other vendor builds do not necessarily ship identical collector sets.

JAVA / COMMAND / SCENARIO EXAMPLE

```
java -XX:+UseShenandoahGC -version
# Supported build: starts; unsupported build: rejects.
```

Why it matters: A vendor choice can change the feasible collector experiment.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you keep deployment images consistent during a Shenandoah rollout?

Answer: Pin reviewed JDK images by digest, verify -XX:+UseShenandoahGC at startup, test each architecture and roll out the same approved image.

> **Interview trap:** OpenJDK feature availability is not proof that every packaged JDK includes it.

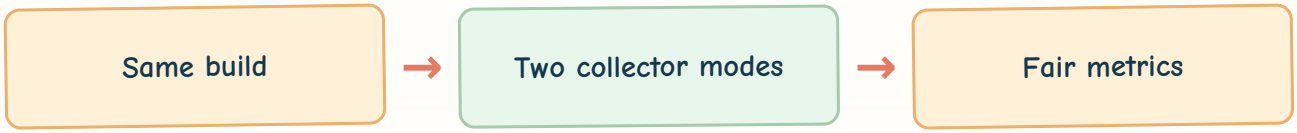
Source guide p. 33 · Technical references J11-J15, J40

Q050. What changed for Generational Shenandoah in Java 25?

SHORT ANSWER · SAY THIS FIRST

JDK 25 supports productized generational Shenandoah on builds that include it; choose the mode explicitly.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

JEP 521 made the generational mode a product feature in JDK 25 after its experimental introduction in JDK 24. On a supporting build, select the documented mode explicitly and verify it; do not assume a mode change only because the JDK version changed.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Supporting JDK 25:
-XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational
Check vendor build and effective runtime behavior.
```

Why it matters: Version and mode belong in the test report.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you compare single-generation and generational modes fairly?

Answer: Use the same JDK build, heap, workload, CPU quota and sample size; change only collector mode and compare real pauses, RSS, CPU and throughput.

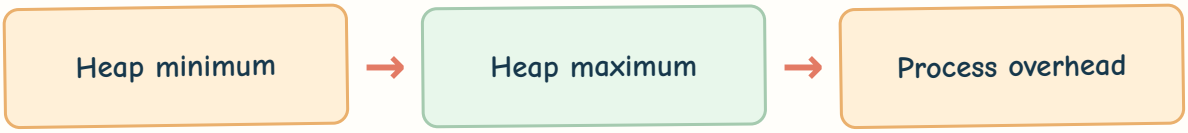
Interview trap: Java 17 execution cannot validate Java 25 Generational Shenandoah behavior.

Source guide p. 34 · Technical references J11-J15, J40

CHAPTER 11 / 20

Heap sizing and OutOfMemoryError

Identify which memory boundary actually failed.



FIVE ORIGINAL INTERVIEW QUESTIONS

051. How do Xms and Xmx differ?

052. What kinds of OutOfMemoryError should you distinguish?

053. Does increasing heap always improve performance?

054. What does GC overhead limit exceeded mean?

055. How do you capture evidence around heap exhaustion?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q051 · Heap sizing and OutOfMemoryError

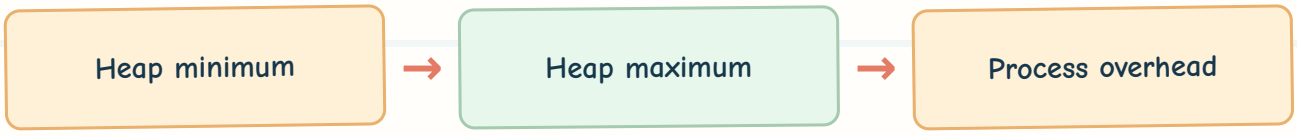
INTERVIEW ANSWER CARD

Q051. How do Xms and Xmx differ?

SHORT ANSWER · SAY THIS FIRST

> Xms sets the starting/minimum heap and Xmx sets its maximum; neither limits the whole process.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Xms controls the initial/minimum heap sizing contract in HotSpot and Xmx controls the maximum heap. Neither caps total process memory. Equal values can reduce heap resizing, but increase baseline commitment expectations and reduce footprint flexibility.

JAVA / COMMAND / SCENARIO EXAMPLE

```
java -Xms256m -Xmx1g Application
# Initial/minimum and maximum object-heap settings.
# Native allocations and thread stacks are additional.
```

Why it matters: Collector behavior and OS page residency still affect physical memory.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

When might equal Xms and Xmx be useful, and what does it cost?

Answer: Equal Xms/Xmx can help predictable heap sizing after warmup but holds a larger heap commitment and reduces memory flexibility.

Interview trap: Xmx is not equivalent to RSS or a container memory limit.

Source guide p. 35 · Technical references J04, J06, J23, J24

Q052 · Heap sizing and OutOfMemoryError

INTERVIEW ANSWER CARD

Q052. What kinds of OutOfMemoryError should you distinguish?

SHORT ANSWER · SAY THIS FIRST

> Java memory errors differ by exhausted resource: heap, metadata, direct buffers or native threads.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

JAVA / COMMAND / SCENARIO EXAMPLE

```
s include Java heap exhaustion, metaspace exhaustion, direct-buffer limits and
inability to create native threads. The message and context guide diagnosis. Some
resource failures or OS kills happen without a useful Java error or dump.
Example
Java heap space -> object allocation/live-set evidence
Metaspace -> class metadata and loader lifetime
Direct buffer memory -> direct allocation budget
Native thread creation -> limits and memory resources
```

Why it matters: Each category has a different ownership and measurement path.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What evidence would you collect for a native-thread OOME?

Answer: Check the exact exception, thread count, ulimit/pid limits, process RSS, native thread-stack use and OS/container events.

Interview trap: Raising Xmx can reduce available native headroom and worsen thread creation failures.

Source guide p. 36 · Technical references J04, J06, J23, J24

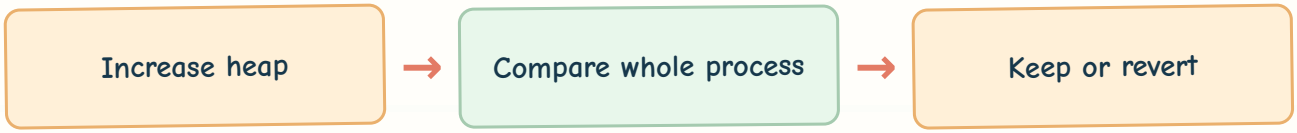
Q053 · Heap sizing and OutOfMemoryError — INTERVIEW ANSWER CARD

Q053. Does increasing heap always improve performance?

SHORT ANSWER · SAY THIS FIRST

More heap can postpone GC, but can increase memory use, pressure elsewhere and some pause costs.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

No. More heap can reduce collection frequency and provide headroom, but can increase footprint, affect layout or work per cycle, and violate process limits. Compare useful throughput and tail latency within the same deployment budget.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Hypothesis: too little allocation headroom
Experiment: modest heap increase within process budget
Compare: GC work, p99, RSS and throttling
```

Why it matters: A larger heap is a capacity change that needs evidence.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What result would make you reject a heap increase?

Answer: Reject it if tail latency, total RSS, container OOM risk or throughput becomes worse, even if GC events become less frequent.

Interview trap: An unbounded live set eventually consumes a larger heap too.

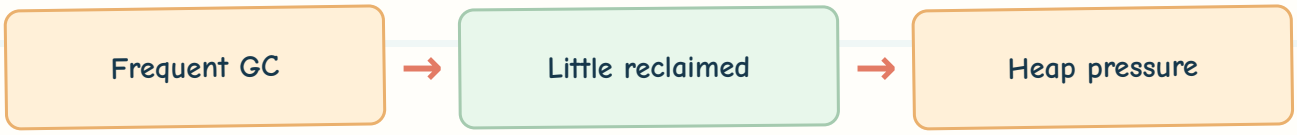
Source guide p. 36 · Technical references J04, J06, J23, J24

Q054. What does GC overhead limit exceeded mean?

SHORT ANSWER · SAY THIS FIRST

GC overhead limit exceeded means the VM spends excessive time collecting but recovers little memory under its policy.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

On collectors and configurations that enforce it, the JVM detects excessive time spent collecting with insufficient reclamation. It is a symptom of an unsustainable memory/workload situation. Investigate retained data and capacity instead of merely disabling the check.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Repeated costly collections
Little useful reclamation
Application makes inadequate progress
```

Why it matters: The exception reports poor progress rather than naming the retaining owner.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you compare the live set with the allowed heap?

Answer: Compare post-full-collection occupancy with Xmx over repeated comparable periods; near-capacity retained data suggests a live-set or leak issue.

Interview trap: Disabling the overhead limit does not create memory or repair retention.

Source guide p. 36 · Technical references J04, J06, J23, J24

Q055 · Heap sizing and OutOfMemoryError — INTERVIEW ANSWER CARD

Q055. How do you capture evidence around heap exhaustion?

SHORT ANSWER · SAY THIS FIRST

> Capture logs, heap evidence and process/container events before failures when safe; guard sensitive dumps.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Use appropriate GC logs and a reviewed heap-dump policy, correlate process/container termination and retain deployment identity. Heap dumps can be large, disruptive and sensitive. Store them securely on a path with capacity rather than assuming every failure produces one.

JAVA / COMMAND / SCENARIO EXAMPLE

```
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/controlled/path
-Xlog:gc*:file=gc.log:uptime,level,tags
```

Why it matters: A Java heap OOME and an OS/container kill have different evidence paths.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you manage dump storage for many replicas?

Answer: Use per-instance bounded dump paths, disk quotas/retention, centralized restricted storage and a tested cleanup/collection plan.

Interview trap: A heap-dump-on-OOME flag does not guarantee a dump after an external kill.

Source guide p. 37 · Technical references J04, J06, J23, J24

CHAPTER 12 / 20

GC logs and quantitative interpretation

Read timings, bytes and long-term trends together.



FIVE ORIGINAL INTERVIEW QUESTIONS

056. How do you enable modern GC and safepoint logging?

057. How do you read heap before/after and pause duration?

058. How can GC duty fraction and allocation rate help?

059. What do User, Sys and Real mean in GC timing?

060. What is a safepoint and is every safepoint a GC pause?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q056 · GC logs and quantitative interpretation

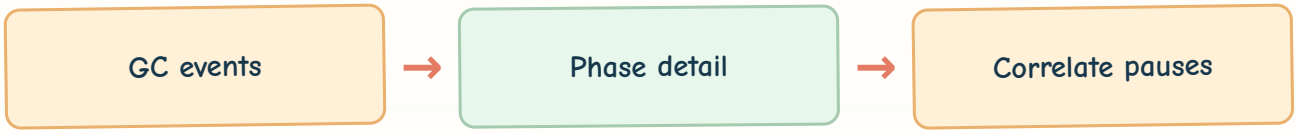
INTERVIEW ANSWER CARD

Q056. How do you enable modern GC and safepoint logging?

SHORT ANSWER · SAY THIS FIRST

Use unified -Xlog settings for GC and safepoints in supported HotSpot versions.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Use unified logging on modern HotSpot. Select tags, level, decorators and output rotation appropriate to the investigation. Verify support on the actual JDK and ensure logs do not create uncontrolled storage or I/O overhead.

JAVA / COMMAND / SCENARIO EXAMPLE

```
-Xlog:gc*,safepoint:file=gc.log:uptime,level,tags:
  filecount=5,filesize=10M
# Join as one option; shown across lines for reading.
```

Why it matters: Decorators and GC identifiers help connect phases belonging to one cycle.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which extra phase detail would you enable for a long G1 evacuation pause?

Answer: Enable focused G1 phase tags such as gc+phases at debug level for a short window; measure logging overhead and disk growth.

Interview trap: Old PrintGCDetails-style recipes do not map unchanged to every modern release.

Source guide p. 38 · Technical references J04, J06, J09, J10, J19

Q057 · GC logs and quantitative interpretation

INTERVIEW ANSWER CARD

Q057. How do you read heap before/after and pause duration?

SHORT ANSWER · SAY THIS FIRST

GC logs show heap occupied before/after events and how long the reported phase took.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Identify the event, cause, heap values and unit, then compare across equivalent workload phases. A before-to-after decrease describes reclamation during that event, not a standalone proof of leak absence or an exact retained-live-set measurement.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Illustrative: GC(7) Pause Young ... 96M->24M(128M) 8.0ms
Before: 96M; after: 24M; shown heap capacity: 128M
Duration: 8.0ms for this reported event
```

Why it matters: A young event can leave garbage in uncollected old regions.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which trends would suggest increasing long-term retention?

Answer: Watch post-GC occupancy across similar traffic windows; an upward baseline after major reclamation suggests retained growth.

Interview trap: Treating every after-young value as the complete live set is incorrect.

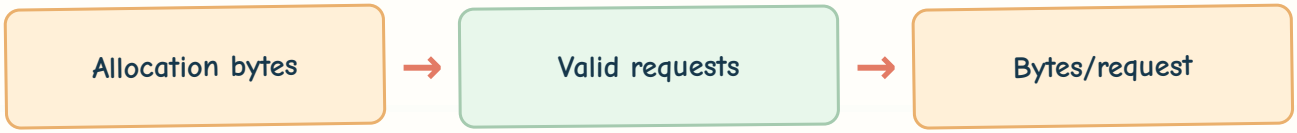
Source guide p. 38 · Technical references J04, J06, J09, J10, J19

Q058. How can GC duty fraction and allocation rate help?

SHORT ANSWER · SAY THIS FIRST

GC duty fraction estimates the share of an interval spent in pauses; allocation rate estimates new object bytes per time.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Compute event time and allocation over a defined observation window with clear counting rules. They expose memory-management demand but must be paired with collector CPU and application metrics. Concurrent cycle wall time is not an STW pause.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Example: 0.5 s STW time / 10 s wall window = 5%
Allocation: 200 MiB over 2 s = 100 MiB/s
Separate: concurrent GC CPU and cycle duration
```

Why it matters: The arithmetic is useful only when the selected measurements match the question.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you estimate bytes allocated per successful request?

Answer: Combine allocation bytes in JFR or profiler with successful request count over the same window; exclude background work or report it separately.

Interview trap: Summing concurrent cycle duration as application stopped time inflates a pause estimate.

Source guide p. 39 · Technical references J04, J06, J09, J10, J19

Q059 · GC logs and quantitative interpretation

INTERVIEW ANSWER CARD

Q059. What do User, Sys and Real mean in GC timing?

SHORT ANSWER · SAY THIS FIRST

User/Sys are CPU time consumed in different modes; Real is elapsed clock time, including scheduling delays.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

They report CPU and elapsed-time perspectives for the event. Parallel worker CPU time can exceed elapsed time. High wall time with limited CPU progress can suggest scheduling pressure; high system time can motivate OS memory/I/O investigation.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Illustrative: User=0.08s Sys=0.01s Real=0.03s
CPU time can sum across workers
Wall time reflects elapsed duration
```

Why it matters: These values help form a hypothesis, not a complete diagnosis by themselves.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What other evidence would confirm CPU throttling?

Answer: Look at cgroup throttled time/periods, container CPU usage versus quota, runnable queues and GC wall-clock-to-CPU gaps.

Interview trap: User plus Sys need not equal Real for parallel execution.

Source guide p. 39 · Technical references J04, J06, J09, J10, J19

Q060 · GC logs and quantitative interpretation

INTERVIEW ANSWER CARD

Q060. What is a safepoint and is every safepoint a GC pause?

SHORT ANSWER · SAY THIS FIRST

> A safepoint is a JVM coordination state; many events besides GC can request one.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A safepoint is a VM coordination state used for operations requiring participating threads to reach suitable execution states. GC is one use, but other operations can require safepoints. Time to reach coordination can differ from time executing the operation.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Request coordination -> threads reach suitable state
VM operation executes -> application resumes
Log: inspect reaching and operation duration
```

Why it matters: A latency spike can involve coordination delay rather than a long collector phase.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you investigate a long time-to-safepoint?

Answer: Enable safepoint logs and inspect time-to-safepoint, busy thread states, native transitions and OS scheduling/CPU starvation.

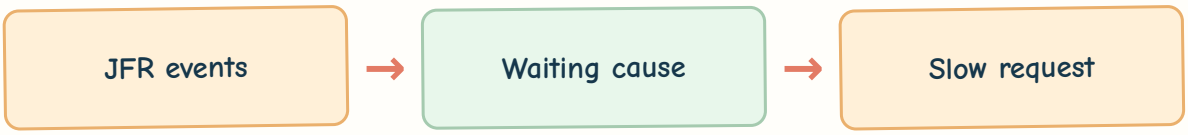
Interview trap: Every application stall is not visible as a long GC pause.

Source guide p. 39 · Technical references J04, J06, J09, J10, J19

CHAPTER 13 / 20

JFR, dumps and native-memory tools

Match each diagnostic tool to the question it answers.



FIVE ORIGINAL INTERVIEW QUESTIONS

061. What does Java Flight Recorder provide?

062. What can a thread dump tell you?

063. What is the difference between a histogram and a heap dump?

064. What does Native Memory Tracking measure?

065. How do CPU and allocation profiles differ?

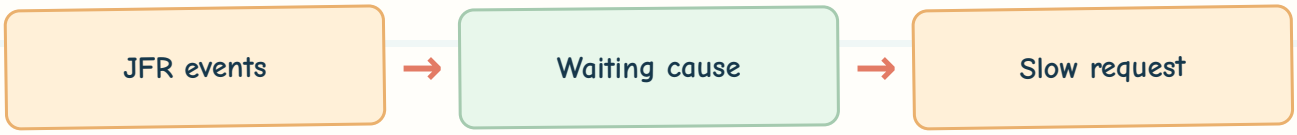
> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q061. What does Java Flight Recorder provide?

SHORT ANSWER · SAY THIS FIRST

JFR records sampled and event-based JVM/application behavior with relatively low overhead.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

JFR records selected runtime and application events for time-correlated diagnosis. Event settings, thresholds and sampling affect what is observed and its cost. Start with a bounded recording and inspect CPU, allocation, locks, I/O and GC according to the question.

JAVA / COMMAND / SCENARIO EXAMPLE

```
jcmd PID JFR.start name=investigation settings=profile \
  duration=60s filename=investigation.jfr
```

Why it matters: A recording links multiple signals within one time window.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which events would you inspect for a slow service with low CPU?

Answer: Inspect socket/file I/O delays, thread parks, monitor contention, blocked times and request-specific spans or custom events.

Interview trap: JFR does not automatically record every method call or every allocation.

Source guide p. 40 · Technical references J21-J24, J28, J29

Q062 · JFR, dumps and native-memory tools INTERVIEW ANSWER CARD

Q062. What can a thread dump tell you?

SHORT ANSWER · SAY THIS FIRST

A thread dump shows what Java threads are doing, including stacks, waits and some lock ownership.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A thread dump gives stacks and state information at a point in time. Repeated dumps can expose stable blocked or waiting patterns and hot loops. Use relevant APIs or tools for deadlock detection and distinguish observations from causal conclusions.

JAVA / COMMAND / SCENARIO EXAMPLE

```
jcmd PID Thread.print -l
# Repeat at meaningful intervals if appropriate.
```

Why it matters: A persistent stack pattern is often more informative than one isolated snapshot.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you connect a Linux hot thread with a Java stack?

Answer: Find the Linux TID using top -H, convert it to hexadecimal and match nid in a JVM thread dump for the same process/time.

Interview trap: RUNNABLE can include native I/O activity; it is not proof of active CPU consumption.

Source guide p. 41 · Technical references J21-J24, J28, J29

Q063. What is the difference between a histogram and a heap dump?

SHORT ANSWER · SAY THIS FIRST

A histogram counts objects by type and size; a heap dump also preserves reference graphs and retaining paths.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A histogram groups object counts and sizes by class. A heap dump contains a graph useful for retained-size and root-path analysis. Diagnostic collection behavior and impact depend on the command and options; a live-oriented request may trigger GC.

JAVA / COMMAND / SCENARIO EXAMPLE

```
jcmd PID GC.class_histogram
jcmd PID help GC.heap_dump
# Review command impact before collecting a large dump.
```

Why it matters: High shallow bytes point to a class; retention analysis identifies an owner.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Why might an empty collection object retain a large backing array?

Answer: Removing entries may not shrink the internal array capacity; the empty collection can still own a large backing array.

Interview trap: The largest class by shallow size is not necessarily the leak's retaining cause.

Source guide p. 41 · Technical references J21-J24, J28, J29

Q064 · JFR, dumps and native-memory tools

INTERVIEW ANSWER CARD

Q064. What does Native Memory Tracking measure?

SHORT ANSWER · SAY THIS FIRST

> Native Memory Tracking reports categorized HotSpot native allocations when enabled; it does not explain all process RSS.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

HotSpot NMT tracks supported internal native allocations when enabled at startup. It can show reserved/committed categories and changes. It does not account for all third-party native allocations or equal OS RSS; reconcile it with other process and platform measurements.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Start: -XX:NativeMemoryTracking=summary
jcmd PID VM.native_memory baseline
jcmd PID VM.native_memory summary.diff scale=MB
```

Why it matters: A diff can locate growth in tracked categories over an interval.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would you inspect if RSS grows while NMT categories stay stable?

Answer: Inspect file mappings, malloc allocator caching, mapped files, shared libraries and system/container memory accounting.

Interview trap: NMT committed memory is not a direct physical-residency measurement.

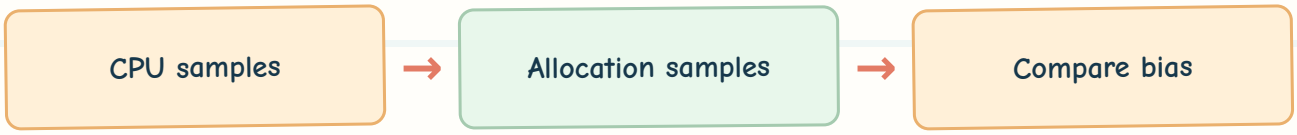
Source guide p. 41 · Technical references J21-J24, J28, J29

Q065. How do CPU and allocation profiles differ?

SHORT ANSWER · SAY THIS FIRST

> CPU profiles show where CPU is spent; allocation profiles show where new objects are created.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A CPU profile samples where active execution spends time. An allocation profile identifies selected allocation sites and rates. A wall-clock or blocking view helps when threads wait. Choose the evidence according to whether work, memory demand or waiting dominates.

JAVA / COMMAND / SCENARIO EXAMPLE

```
High CPU -> sampled hot execution paths
High allocation -> object creation sites
Low CPU, high latency -> waits, queues, dependencies
```

Why it matters: An expensive allocation site may be a useful optimization even if it is not the top CPU frame.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you evaluate profiler overhead and sampling bias?

Answer: Compare sampling frequency, event drop rate, run-to-run noise and service overhead; use representative loads and corroborating metrics.

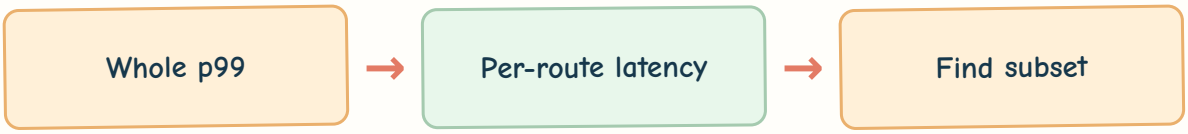
> **Interview trap:** A CPU profile alone can miss a database wait dominating request latency.

Source guide p. 42 · Technical references J21-J24, J28, J29

CHAPTER 14 / 20

CPU, locks and tail latency

Find CPU, locks and queueing before blaming GC.



FIVE ORIGINAL INTERVIEW QUESTIONS

066. Why can p99 rise when average latency stays low?

067. What is the difference between service time and queueing time?

068. How do you diagnose lock contention?

069. Why is high CPU not always a GC problem?

070. How can retries create performance collapse?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q066 · CPU, locks and tail latency

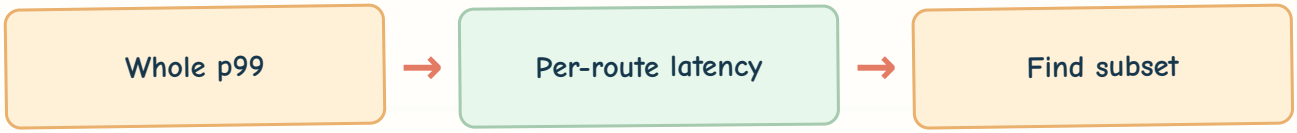
INTERVIEW ANSWER CARD

Q066. Why can p99 rise when average latency stays low?

SHORT ANSWER · SAY THIS FIRST

p99 captures the slowest user requests; a small slow group can increase p99 while mean stays stable.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A small subset of requests can encounter pauses, queueing, slow dependencies, contention or cold execution. An average dilutes those outliers. Measure percentiles with enough observations and a defined population, then correlate the slow path.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Most requests: 20 ms
Small tail: 1,000 ms
Average can look acceptable while p99 is poor
```

Why it matters: Endpoint and tenant segmentation can reveal a concentrated problem.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you separate one slow endpoint from a whole-service regression?

Answer: Break down histograms by endpoint, status and region; compare each route's traffic share and dependency timing.

Interview trap: p99 estimates from a tiny sample are not stable evidence.

Source guide p. 43 · Technical references J21, J22, J28, J38

Q067 · CPU, locks and tail latency

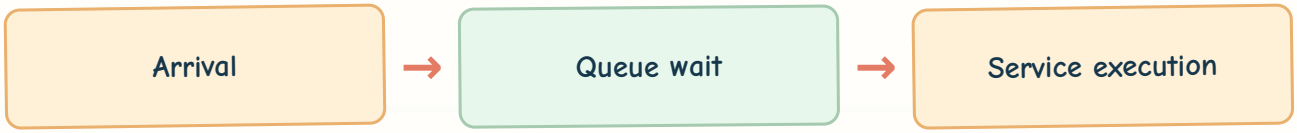
INTERVIEW ANSWER CARD

Q067. What is the difference between service time and queueing time?

SHORT ANSWER · SAY THIS FIRST

> Service time is active work; queueing time is waiting before work or a scarce resource.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Service time is time spent executing the required work, while queueing time is time waiting for admission or resources. As utilization approaches a bottleneck, waiting can grow sharply. Instrument acquisition and execution separately.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Request total = admission + pool wait + work + I/O
JDBC acquisition: 400 ms
SQL execution: 10 ms
```

Why it matters: Optimizing the 10 ms SQL step alone may miss the dominant pool delay.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which pool and dependency metrics would you correlate?

Answer: Correlate executor queue age, Hikari pending/acquisition duration, HTTP client pool waits, DB query time and arrival rate.

> **Interview trap:** More threads can increase contention and waiting rather than throughput.

Source guide p. 44 · Technical references J21, J22, J28, J38

Q068 · CPU, locks and tail latency

INTERVIEW ANSWER CARD

Q068. How do you diagnose lock contention?

SHORT ANSWER · SAY THIS FIRST

> Lock contention makes threads wait for exclusive access even when application code is otherwise fast.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Use repeated stacks and lock-related recording events to identify waiting threads, owners and critical sections. Then reduce unnecessary lock scope or contention while preserving correctness. Evaluate throughput and tail latency under the same concurrency.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Many requests -> one synchronized critical section
Owner -> slow I/O while holding lock
Candidate repair -> separate I/O from guarded state
```

Why it matters: The right change depends on the protected invariant.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you verify the correctness of a lock-scope reduction?

Answer: Add concurrency tests for invariants and race conditions, then measure contention and p99 before and after the narrower lock.

Interview trap: Replacing a lock with a concurrent collection does not preserve every multi-step atomic operation.

Source guide p. 44 · Technical references J21, J22, J28, J38

Q069 · CPU, locks and tail latency

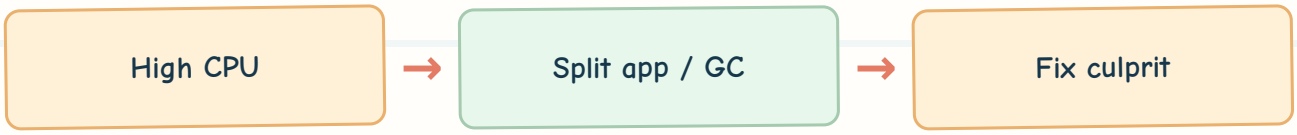
INTERVIEW ANSWER CARD

Q069. Why is high CPU not always a GC problem?

SHORT ANSWER · SAY THIS FIRST

> High CPU can come from business loops, serialization, retries, spinning or GC; profile before guessing.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

CPU can be consumed by application loops, serialization, cryptography, compilation or collector work. Use CPU sampling, GC events and per-thread evidence to identify the owner. Diagnose the hot path before changing collector settings.

JAVA / COMMAND / SCENARIO EXAMPLE

```
CPU: high
GC pause fraction: low
Profile: repeated JSON serialization dominates
Hypothesis: reduce redundant serialization work
```

Why it matters: A collector experiment is useful only if it addresses a measured contribution.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you quantify concurrent collector CPU versus application CPU?

Answer: Use JFR/CPU profiles and process CPU; compare GC worker activity in logs or OS thread samples against application hot stacks.

Interview trap: The coexistence of GC activity and high CPU does not prove GC is the dominant cost.

Source guide p. 44 · Technical references J21, J22, J28, J38

Q070 · CPU, locks and tail latency

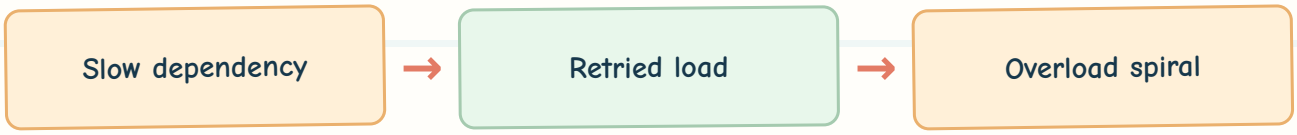
INTERVIEW ANSWER CARD

Q070. How can retries create performance collapse?

SHORT ANSWER · SAY THIS FIRST

> Retries multiply load when a dependency is slow, potentially creating a positive feedback loop.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Retries multiply offered load when a dependency is already slow. Extra waiting requests retain state and consume pools, threads and memory. Use deadlines, bounded attempts, backoff/jitter and concurrency limits with idempotent or resolvable operations.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Client retries x service retries x SDK retries
Effective attempts can multiply
Budget: end-to-end deadline and bounded admission
```

Why it matters: The memory and latency symptom may originate from failure policy rather than GC tuning.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would you cap first during a dependency incident?

Answer: Cap in-flight dependency calls and retry attempts first, apply deadlines and jittered backoff, and shed noncritical work.

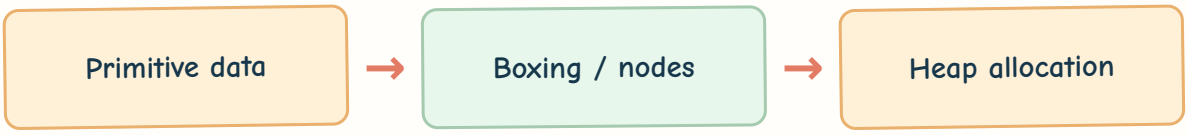
> **Interview trap:** An unlimited retry queue can become an unbounded retained object graph.

Source guide p. 45 · Technical references J21, J22, J28, J38

CHAPTER 15 / 20

Allocation-aware Java coding

Reduce useful allocation cost, not every object blindly.



FIVE ORIGINAL INTERVIEW QUESTIONS

071. How do boxing and collections affect allocation?

072. How can string and logging work create allocation pressure?

073. Should every temporary object be pooled?

074. How do you reduce large-payload memory spikes?

075. Are parallel streams always faster?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q071 · Allocation-aware Java coding

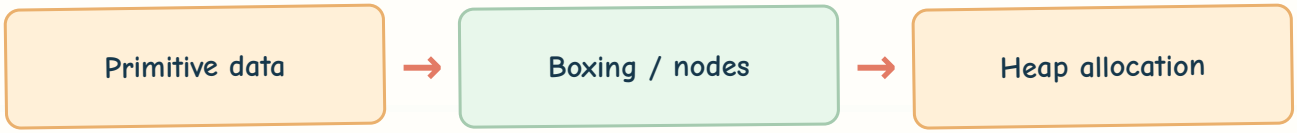
INTERVIEW ANSWER CARD

Q071. How do boxing and collections affect allocation?

SHORT ANSWER · SAY THIS FIRST

> Boxed values and collection nodes allocate extra objects compared with many primitive representations.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Boxing and reference-based data structures can add object and pointer overhead, though caching and optimization change exact allocation. Primitive arrays or specialized approaches can help measured hot paths. Compare semantics, workload and retained footprint.

JAVA / COMMAND / SCENARIO EXAMPLE

```
List<Integer> -> references and possible boxed values
int[] -> primitive payload in one array
IntStream -> primitive operations where appropriate
```

Why it matters: The data representation can matter more than a collector flag.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you measure the allocation cost of a real collection workload?

Answer: Profile allocation bytes and objects per operation on realistic inputs; validate with JMH where helpful and end-to-end behavior.

Interview trap: Not every boxing expression necessarily creates a fresh object.

Source guide p. 46 · Technical references J26, J30-J32

Q072 · Allocation-aware Java coding

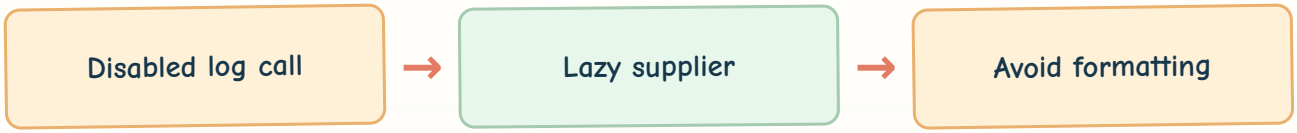
INTERVIEW ANSWER CARD

Q072. How can string and logging work create allocation pressure?

SHORT ANSWER · SAY THIS FIRST

> Formatting, concatenating and logging strings can allocate many short-lived objects.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Repeated formatting, concatenation or payload construction can allocate substantial temporary data. Parameterized logging can avoid some formatting when disabled, but argument expressions still evaluate. Guard expensive computation where justified and keep sensitive payloads out of logs.

JAVA / COMMAND / SCENARIO EXAMPLE

```
log.debug("value={}", cheapValue);
if (log.isDebugEnabled()) {
 log.debug("summary={}", buildExpensiveSummary());
}
```

Why it matters: The logging API does not automatically defer every method argument.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

When would a logging supplier API be useful?

Answer: Supplier-style lazy messages help when disabled log levels would otherwise construct expensive data; avoid capturing large objects unnecessarily.

Interview trap: Enabling verbose payload logs during load can change the performance being diagnosed.

Source guide p. 46 · Technical references J26, J30-J32

Q073 · Allocation-aware Java coding

INTERVIEW ANSWER CARD

Q073. Should every temporary object be pooled?

SHORT ANSWER · SAY THIS FIRST

> Pooling small objects can add state, contention and retention costs; measure first.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

No. Ordinary allocation can be cheap and the collector may handle short-lived objects efficiently. Pools introduce retained memory, ownership, concurrency and reset complexity. Pool only a measured expensive resource or representation with a clear bounded lifecycle.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Cheap short-lived DTO -> ordinary allocation
Connection/socket -> managed bounded lifecycle
Pooled buffer -> explicit ownership and size bound
```

Why it matters: A pool can convert short-lived data into a long-lived retained graph.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would make a buffer pool worth its complexity?

Answer: A buffer pool helps if allocation/GC cost is material, reuse is frequent, and bounded pool size improves actual throughput/latency.

Interview trap: Reusing an object without clearing request-specific state can leak data between users.

Source guide p. 47 · Technical references J26, J30-J32

Q074 · Allocation-aware Java coding

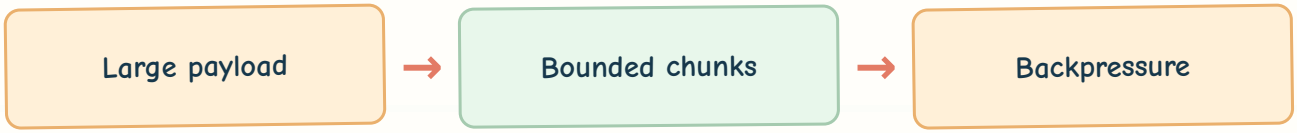
INTERVIEW ANSWER CARD

Q074. How do you reduce large-payload memory spikes?

SHORT ANSWER · SAY THIS FIRST

> Stream large data in bounded pieces rather than holding whole files or payloads in heap.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Use bounded streaming, pagination, chunking and admission limits rather than buffering every payload or result at once. Ensure downstream APIs and transaction behavior support the intended stream. Measure peak concurrent memory, not just one request.

JAVA / COMMAND / SCENARIO EXAMPLE

```
100 requests x 10 MiB full buffers = about 1,000 MiB
Bounded chunk processing -> different peak profile
Limit concurrency -> bound simultaneous payload state
```

Why it matters: Per-request memory multiplies across concurrent requests.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you test a slow consumer without unbounded buffering?

Answer: Throttle a test client/consumer, limit producer buffering and verify stable RSS, bounded queue length and correct cancellation.

Interview trap: A Stream API expression does not guarantee end-to-end streaming if the source was already fully materialized.

Source guide p. 47 · Technical references J26, J30-J32

Q075 · Allocation-aware Java coding

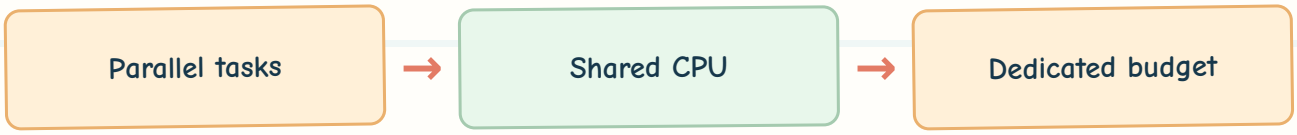
INTERVIEW ANSWER CARD

Q075. Are parallel streams always faster?

SHORT ANSWER · SAY THIS FIRST

> Parallel streams use shared parallel execution resources and can hurt small tasks or request fairness.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

No. Parallel work adds scheduling, coordination and resource contention. Benefit depends on work size, independence and available CPU. Blocking I/O and shared pools require particular care. Measure useful throughput and service interference rather than only one loop duration.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Small cheap map -> overhead may dominate
Large independent CPU work -> candidate for testing
Blocking request path -> inspect pool/resource behavior
```

Why it matters: A shared execution resource can couple unrelated workloads.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you isolate CPU-heavy background work from HTTP request capacity?

Answer: Use a dedicated bounded executor or explicit concurrency control for CPU-heavy background work, separate from HTTP request capacity.

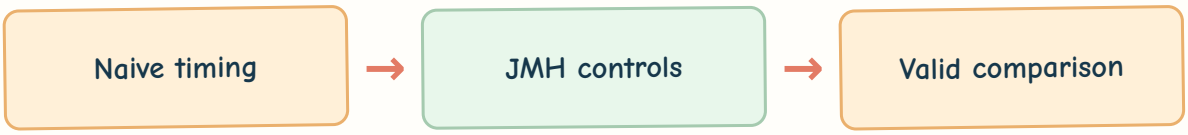
Interview trap: Parallelizing an operation does not make its shared mutable state safe.

Source guide p. 47 · Technical references J26, J30-J32

CHAPTER 16 / 20

Benchmarking and experiment design

Prevent compiler optimizations from invalidating tests.



FIVE ORIGINAL INTERVIEW QUESTIONS

076. Why are naive nanoTime microbenchmarks unreliable?

077. How does dead-code elimination affect a benchmark?

078. What is constant folding in a benchmark?

079. Why do JMH warmup and forks matter?

080. How do you design a fair collector experiment?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q076 · Benchmarking and experiment design

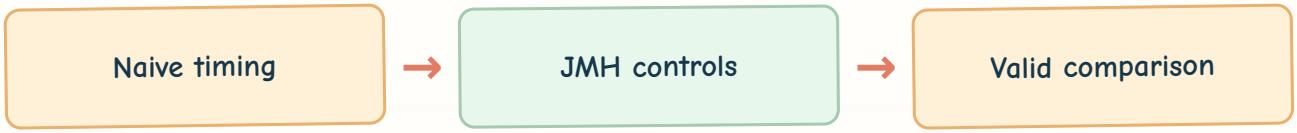
INTERVIEW ANSWER CARD

Q076. Why are naive nanoTime microbenchmarks unreliable?

SHORT ANSWER · SAY THIS FIRST

A tiny loop using nanoTime is affected by JIT warmup, dead-code removal and measurement noise.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Compilation, dead-code elimination, constant folding, timer cost and environmental noise can dominate an ad hoc loop. JMH provides a structured harness, but its setup must still model the question. A microbenchmark does not replace an end-to-end load test.

JAVA / COMMAND / SCENARIO EXAMPLE

```
long start = System.nanoTime();
// Tiny loop with unused result: misleading experiment.
long elapsed = System.nanoTime() - start;
```

Why it matters: The measured program might not execute the work you intended.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would you change before claiming one collection implementation is faster?

Answer: Use JMH with controlled inputs, warmups, forks and allocation profiling; compare confidence intervals and actual service performance.

Interview trap: A precise timer does not make a flawed experiment accurate.

Source guide p. 48 · Technical references J30-J33

Q077 · Benchmarking and experiment design

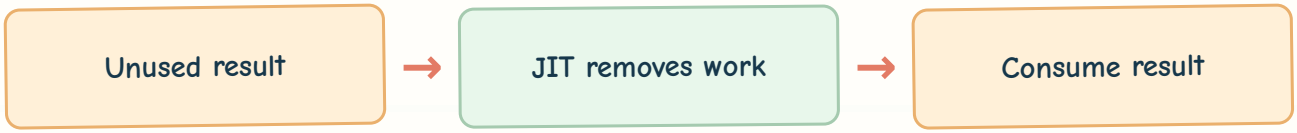
INTERVIEW ANSWER CARD

Q077. How does dead-code elimination affect a benchmark?

SHORT ANSWER · SAY THIS FIRST

The compiler may eliminate work if its result cannot be observed, making timings unrealistically fast.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

If a result has no relevant observable use, optimized code may remove all or part of the computation. Return benchmark results or use suitable JMH consumption mechanisms. Inspect generated behavior or allocation evidence when the result seems implausible.

JAVA / COMMAND / SCENARIO EXAMPLE

```
@Benchmark
public int sum() {
 return values[0] + values[1];
}
// JMH consumes the benchmark result.
```

Why it matters: The measured operation needs a meaningful escape to the harness.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you keep an allocation benchmark representative without forcing unrelated overhead?

Answer: Return or consume realistic results via JMH, keep data dependencies, and compare allocation profiles without adding artificial heavy sinks.

Interview trap: An unused allocation loop can benchmark optimized-away work.

Source guide p. 48 · Technical references J30-J33

Q078 · Benchmarking and experiment design

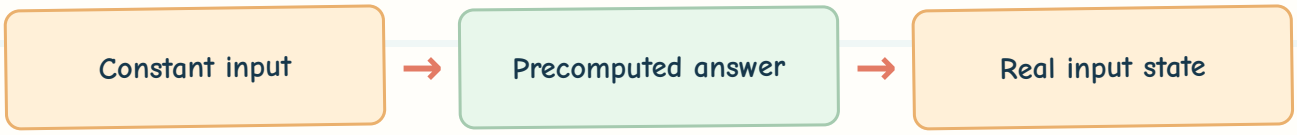
INTERVIEW ANSWER CARD

Q078. What is constant folding in a benchmark?

SHORT ANSWER · SAY THIS FIRST

> The compiler can precompute constant expressions, so your benchmark might avoid the intended operation.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

The compiler can precompute operations on inputs it can prove constant. Use appropriately managed state and realistic varying inputs when measuring runtime work. Do not confuse Java field modifiers with the full compiler's ability to infer constants.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Constant expression -> possible precomputed result
Benchmark state -> representative input source
@Param -> compare controlled workload dimensions
```

Why it matters: A nanosecond result can reflect a constant result rather than the full algorithm.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Where should input preparation occur?

Answer: Prepare input data in JMH setup/State at an appropriate iteration or invocation scope, keeping preparation cost separate from measured work.

Interview trap: Randomizing inputs inside the timed method may instead benchmark random-number generation.

Source guide p. 49 · Technical references J30-J33

Q079 · Benchmarking and experiment design

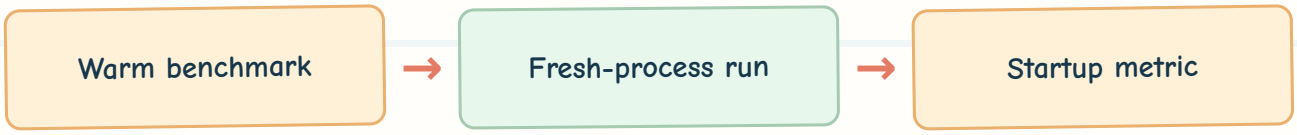
INTERVIEW ANSWER CARD

Q079. Why do JMH warmup and forks matter?

SHORT ANSWER · SAY THIS FIRST

> JMH warmup measures optimized hot paths; separate JVM forks reduce contamination from prior measurements.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Warmup lets the runtime reach a defined execution condition. Forks isolate JVM execution histories and reduce cross-test profile contamination. Choose iterations, forks and measurement modes from the workload and variability; default values are not universal proof of rigor.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Record: JDK, JVM flags, forks, warmup and measurement
Observe: distribution and error, not one best number
Validate: representative inputs and consumed outputs
```

Why it matters: Different execution histories can produce different compiled code.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you compare startup-sensitive operations instead of warmed throughput?

Answer: Measure cold-start-to-ready and first-request latency in fresh processes, with repeatable startup inputs, instead of relying on warmed JMH scores.

Interview trap: Running one benchmark after another in one JVM can couple their profiles.

Source guide p. 49 · Technical references J30-J33

Q080 · Benchmarking and experiment design

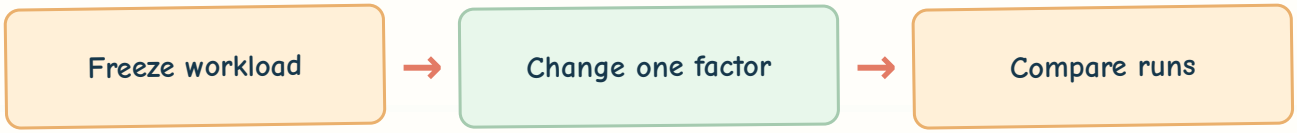
INTERVIEW ANSWER CARD

Q080. How do you design a fair collector experiment?

SHORT ANSWER · SAY THIS FIRST

A collector comparison needs identical workload, JVM build, heap, CPU limits and valid end-to-end metrics.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Hold application version, workload, resource limits and most settings constant; record unavoidable collector-specific differences. Repeat runs, separate warmup from observation and measure the service objective with error rates. Include memory and CPU cost alongside pauses.

JAVA / COMMAND / SCENARIO EXAMPLE

```
A: G1, same image and representative traffic
B: ZGC, same budget and deployment placement
Compare: p99, successful throughput, CPU, RSS, stalls
```

Why it matters: A fair comparison explains any changed variable, including JDK version.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you disentangle a JDK upgrade from a collector change?

Answer: Use a matrix: run old/new JDK each with its supported matching collector/mode, then compare within-JDK and cross-JDK separately.

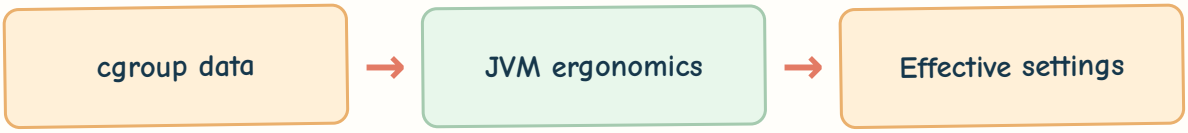
Interview trap: Comparing Java 17 G1 with Java 25 ZGC cannot isolate collector choice alone.

Source guide p. 49 · Technical references J30-J33

CHAPTER 17 / 20

Containers and Kubernetes JVM budgets

Container limits apply to the whole JVM process.



FIVE ORIGINAL INTERVIEW QUESTIONS

081. How does HotSpot become container-aware?

082. How do MaxRAMPercentage and Xmx interact?

083. Why can a pod be OOMKilled while heap usage is below Xmx?

084. How does CPU throttling affect GC and latency?

085. How should you budget process memory?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q081 · Containers and Kubernetes JVM budgets

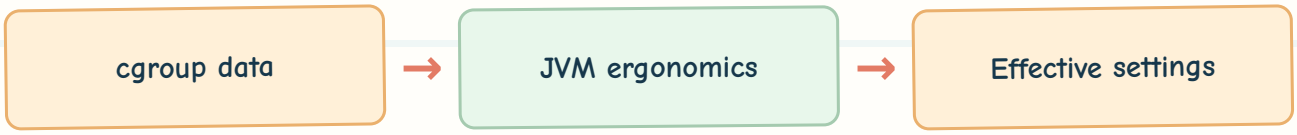
INTERVIEW ANSWER CARD

Q081. How does HotSpot become container-aware?

SHORT ANSWER · SAY THIS FIRST

> Container-aware HotSpot reads applicable cgroup resource limits to choose defaults and ergonomics.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Supported modern HotSpot builds on supported platforms can use container resource information for ergonomics. Verify the detected memory and processors, effective flags and runtime build. Detection and limits differ by OS, cgroup version and deployment configuration.

JAVA / COMMAND / SCENARIO EXAMPLE

```
java -XshowSettings:system -version
java -XX:+PrintCommandLineFlags -version
# Inspect what the actual deployment detects.
```

Why it matters: The same image can choose different ergonomics under different limits.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would you inspect after moving from cgroup v1 to v2?

Answer: Inspect java -XshowSettings:system -version, cgroup files, actual Xmx, availableProcessors and runtime logs after the migration.

Interview trap: Running on a host does not validate behavior in your Kubernetes pod.

Source guide p. 50 · Technical references J04-J06, J23

Q082 · Containers and Kubernetes JVM budgets

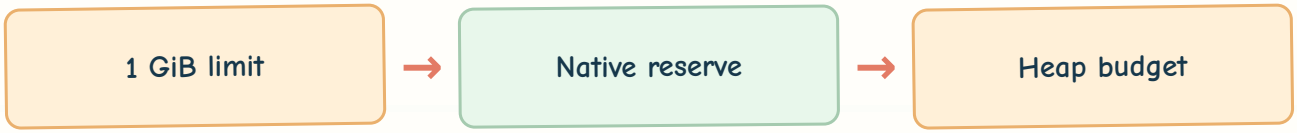
INTERVIEW ANSWER CARD

Q082. How do MaxRAMPercentage and Xmx interact?

SHORT ANSWER · SAY THIS FIRST

MaxRAMPercentage helps choose heap ergonomically when Xmx is not explicitly set; an explicit Xmx wins.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Heap-percentage flags affect ergonomic sizing when applicable; an explicit Xmx sets the heap maximum rather than making the percentage a second independent cap. The denominator depends on effective memory detection and configuration. Inspect the resulting heap.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Option A: -Xmx512m
Option B: -XX:MaxRAMPercentage=50
# Alternatives to evaluate, not equivalent everywhere.
```

Why it matters: Neither option bounds total native memory or guarantees container headroom.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you choose a budget for a 1 GiB Java container?

Answer: Reserve memory for metaspace, thread stacks, code, direct buffers, native libraries and safety headroom; measure RSS before fixing Xmx.

Interview trap: A high percentage may leave too little memory for threads and buffers.

Source guide p. 51 · Technical references J04-J06, J23

Q083 · Containers and Kubernetes JVM budgets

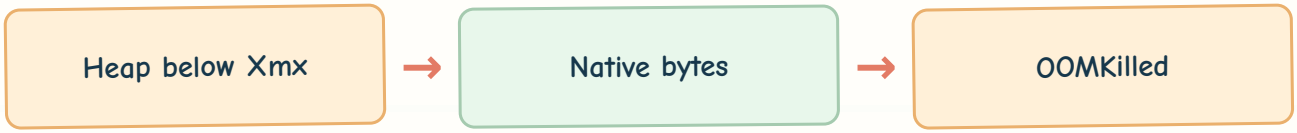
INTERVIEW ANSWER CARD

Q083. Why can a pod be OOMKilled while heap usage is below Xmx?

SHORT ANSWER · SAY THIS FIRST

A container memory kill can happen when native and heap memory together exceed the cgroup limit.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

The platform enforces its own charged-memory limit, which includes more than the Java heap. Native allocations, stacks, direct buffers and other charged resources can exceed that limit. Inspect termination and cgroup evidence alongside JVM metrics.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Heap used: below Xmx
Direct buffers and stacks: additional memory
Platform charged memory: exceeds container limit
Outcome: external kill may produce no heap dump
```

Why it matters: The failing boundary is the container, not necessarily Java heap allocation.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which deployment and process metrics would distinguish this from heap OOME?

Answer: Check termination reason, cgroup OOM events, RSS, heap-after-GC, NMT, direct buffers and thread counts.

Interview trap: Raising Xmx can worsen the external kill risk.

Source guide p. 51 · Technical references J04-J06, J23

Q084 · Containers and Kubernetes JVM budgets

INTERVIEW ANSWER CARD

Q084. How does CPU throttling affect GC and latency?

SHORT ANSWER · SAY THIS FIRST

CPU throttling delays application and GC workers, increasing real time even if GC work itself is unchanged.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

A CPU quota can delay application and collector progress even when the host has spare processors. Concurrent collectors need time to keep up, and parallel phases share the same effective resource constraint. Inspect throttling and runnable demand together with GC timing.

JAVA / COMMAND / SCENARIO EXAMPLE

```
CPU demand > quota -> scheduling delay
Concurrent marking slower -> less reclamation headroom
Request queues -> longer end-to-end latency
```

Why it matters: A collector can appear slower because its environment changed.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you test whether a long pause is primarily scheduling delay?

Answer: Compare GC User+Sys with Real under throttling metrics; repeat at a higher CPU quota and test whether wall-time gaps shrink.

Interview trap: Adding GC threads does not create more CPU quota.

Source guide p. 51 · Technical references J04-J06, J23

Q085 · Containers and Kubernetes JVM budgets

INTERVIEW ANSWER CARD

Q085. How should you budget process memory?

SHORT ANSWER · SAY THIS FIRST

Total Java process memory includes heap, metadata, code cache, native buffers, stacks and overhead.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Start from measured peak charged memory and break down plausible contributors: heap, metadata, code, thread stacks, direct/native buffers and other process/deployment charges. Leave justified headroom and validate under peak and failure traffic. Estimates are not exact additive OS accounting.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Example 1 GiB limit:
512 MiB heap + measured native budget + headroom
Check concurrent requests, thread count and buffers
Validate actual charged-memory peak
```

Why it matters: The budget must survive realistic concurrency, not only startup.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would the budget change for thousands of platform threads?

Answer: Many platform threads consume stacks and scheduling/native-thread resources; multiply observed per-thread costs and include concurrency limits.

Interview trap: A universal 'set heap to 75%' recipe ignores workload-dependent native use.

Source guide p. 52 · Technical references J04-J06, J23

CHAPTER 18 / 20

Spring Boot production tuning

Treat the database pool and downstreams as shared budgets.



FIVE ORIGINAL INTERVIEW QUESTIONS

086. Which metrics should you watch for Spring Boot?

087. How can connection-pool sizing affect JVM performance?

088. How do WebFlux and virtual threads change resource needs?

089. How do you tune startup for autoscaling?

090. How do you roll out a JVM tuning change safely?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q086 · Spring Boot production tuning

INTERVIEW ANSWER CARD

Q086. Which metrics should you watch for Spring Boot?

SHORT ANSWER · SAY THIS FIRST

> Spring Boot needs user latency plus JVM, executor, DB pool and dependency metrics, not only heap graphs.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Track customer rate, errors and latency, plus JVM allocation/GC/memory, executor saturation, connection-pool waits and dependency timing. Use supported metrics integrations and stable labels. Protect operational endpoints and measure overhead of added telemetry.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Customer: successful rate and p99
JVM: allocation, pauses, live-set trends
Pools: queued work and acquisition wait
Dependencies: SQL and outbound latency
```

Why it matters: Framework health and useful customer outcomes are distinct signals.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What metric would identify HikariCP acquisition delay?

Answer: Monitor HikariCP connection acquisition time and pending-connection gauges, with timeout counts and max-pool utilization.

Interview trap: An up health endpoint does not prove requests are fast or transactions succeed.

Source guide p. 53 · Technical references J21-J24, J34, J35, J38, J39

Q087 · Spring Boot production tuning

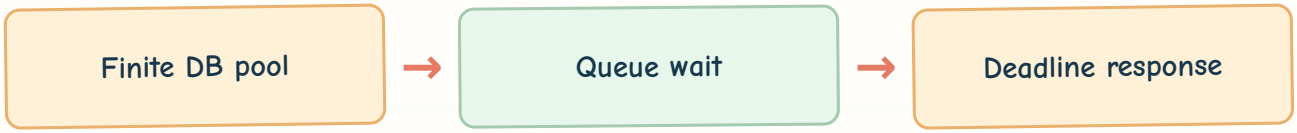
INTERVIEW ANSWER CARD

Q087. How can connection-pool sizing affect JVM performance?

SHORT ANSWER · SAY THIS FIRST

More JDBC connections are not always better; too many can saturate the database and amplify wait costs.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Blocked requests can retain object graphs and occupy platform threads while waiting for a connection. A larger pool may overload the database rather than help. Budget connections across all replicas and instrument waiting separately from SQL execution.

JAVA / COMMAND / SCENARIO EXAMPLE

```
20 replicas x pool maximum 15 = 300 connections
Database capacity and other clients constrain budget
Waiting requests -> retained context and response data
```

Why it matters: The bottleneck can be a database concurrency budget expressed as a JVM memory symptom.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What should happen when the pool is full and the request deadline expires?

Answer: Return a controlled timeout/503 or suitable overload response; release caller resources and do not let requests wait beyond their deadline.

Interview trap: Pool max is multiplied by replicas and can change during autoscaling.

Source guide p. 53 · Technical references J21-J24, J34, J35, J38, J39

Q088 · Spring Boot production tuning

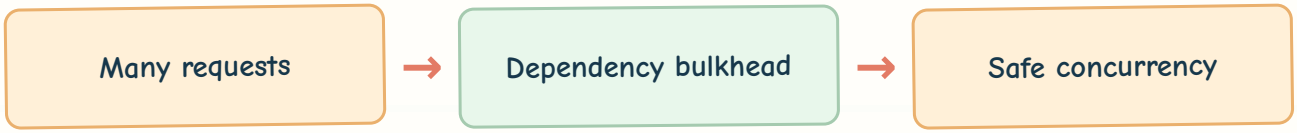
INTERVIEW ANSWER CARD

Q088. How do WebFlux and virtual threads change resource needs?

SHORT ANSWER · SAY THIS FIRST

> WebFlux changes the I/O concurrency model; virtual threads allow blocking-style code with many virtual tasks, but resources remain finite.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

They change how waiting work occupies execution resources, but do not remove downstream limits or per-request state. Avoid blocking event-loop code in reactive applications. With virtual threads, bound resource access and admitted work rather than assuming cheap threads make unlimited concurrency safe.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Reactive: event loop -> nonblocking composition
Virtual thread: simpler blocking style where suitable
Both: DB, remote API and memory budgets still apply
```

Why it matters: The concurrency model changes the cost profile, not the service's business invariants.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which limit would you enforce around a slow external API?

Answer: Use a bulkhead/semaphore on concurrent external calls plus connection limits and timeouts; neither model grants infinite downstream capacity.

> **Interview trap:** More in-flight requests can increase memory even with cheap thread scheduling.

Source guide p. 54 · Technical references J21-J24, J34, J35, J38, J39

Q089 · Spring Boot production tuning

INTERVIEW ANSWER CARD

Q089. How do you tune startup for autoscaling?

SHORT ANSWER · SAY THIS FIRST

> **Autoscaling succeeds only if new JVM instances warm up before traffic overwhelms the existing fleet.**

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Measure time spent in loading, bean creation, database checks, initialization and compilation. Remove unnecessary eager work while keeping readiness truthful. Use supported startup instrumentation and evaluate startup features for your selected JDK and framework version.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Start process -> initialize -> readiness -> traffic
Measure each phase and first-request latency
Avoid moving failures invisibly into customer requests
```

Why it matters: Lazy initialization can shift work and failure from startup into request handling.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you verify that a newly scaled replica is truly useful?

Answer: Replay realistic traffic against newly Ready replicas; monitor startup time, first-request p99, error rate, cache warmup and capacity.

Interview trap: A faster readiness signal that precedes required initialization can worsen service reliability.

Source guide p. 54 · Technical references J21-J24, J34, J35, J38, J39

Q090 · Spring Boot production tuning

INTERVIEW ANSWER CARD

Q090. How do you roll out a JVM tuning change safely?

SHORT ANSWER · SAY THIS FIRST

Tune JVM settings with a baseline, a small controlled change, canary observation and a rollback plan.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Change a small reviewed set of variables, deploy to representative limited traffic and compare customer, GC and resource metrics. Define abort criteria and keep a compatible rollback configuration. Record exact JDK, flags, image and environment.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Hypothesis -> one controlled change -> canary
Observe -> p99, errors, useful rate, RSS, CPU
Decision -> expand or restore prior configuration
```

Why it matters: A flag-only release still changes runtime behavior and resource demand.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would trigger immediate rollback of a heap-size experiment?

Answer: Rollback if user p99, errors, OOM kills, CPU throttling or RSS exceeds predefined safe thresholds.

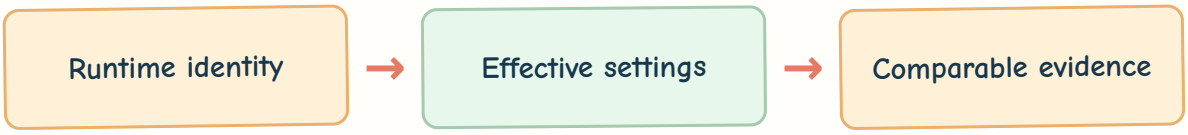
Interview trap: A canary with only idle traffic does not validate a peak-load memory change.

Source guide p. 54 · Technical references J21-J24, J34, J35, J38, J39

CHAPTER 19 / 20

Modern Java version traps

State JDK build, version and exact collector mode.



FIVE ORIGINAL INTERVIEW QUESTIONS

091. What JVM and GC version details should an interview answer include?

092. Do virtual threads eliminate platform threads and memory costs?

093. How did virtual-thread monitor pinning change?

094. What are compact object headers in Java 25?

095. How should you evaluate a JDK upgrade for performance?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q091 · Modern Java version traps

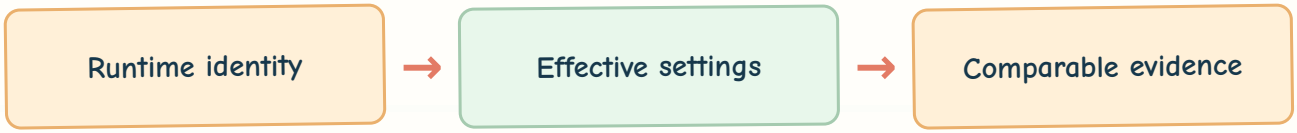
INTERVIEW ANSWER CARD

Q091. What JVM and GC version details should an interview answer include?

SHORT ANSWER · SAY THIS FIRST

> Always state the exact JDK vendor/build, GC name/mode and JVM/container flags when discussing tuning.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

State the JDK release, vendor/build, collector and mode, architecture and relevant resource limits. Identify removed or changed options before migration. Avoid presenting one release's default as a permanent property of the Java language.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Report: Java 17 HotSpot, G1, Linux/x86_64
Alternative: Java 25 supporting build, Generational ZGC
Record exact flags and resource context
```

Why it matters: This makes the answer reproducible and its scope assessable.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which runtime details belong in a production performance report?

Answer: Record java -version, effective flags, heap, container limits, CPU architecture, collector mode, image digest, workload and measurements.

Interview trap: 'We use Java GC' omits the collector contract being discussed.

Source guide p. 55 · Technical references J04-J06, J12-J16, J34-J36

Q092 · Modern Java version traps

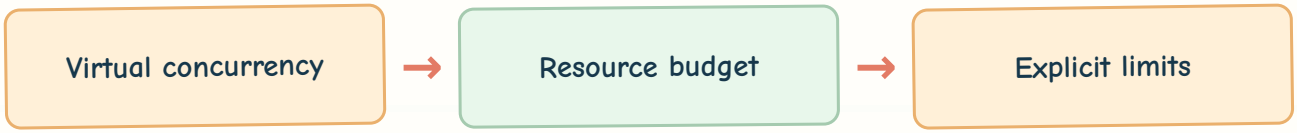
INTERVIEW ANSWER CARD

Q092. Do virtual threads eliminate platform threads and memory costs?

SHORT ANSWER · SAY THIS FIRST

> Virtual threads reduce per-request platform-thread costs but still consume memory and downstream resources.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

No. Virtual threads run on carrier platform threads and retain their own execution state. They can improve scalability for suitable waiting workloads, but cannot make CPU work infinitely parallel or remove memory and dependency budgets. Available since Java 21 as a final feature.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Many waiting tasks -> virtual-thread candidates
CPU-heavy tasks -> limited by actual processors
In-flight state -> still consumes memory
```

Why it matters: Improved thread economics can expose a downstream bottleneck sooner.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you prevent unbounded virtual-thread work from exhausting memory?

Answer: Set admission/semaphore limits, timeouts and bounded request queues; monitor heap, native usage and active external calls.

Interview trap: Virtual threads should not be described as a faster garbage collector.

Source guide p. 55 · Technical references J04-J06, J12-J16, J34-J36

Q093 · Modern Java version traps

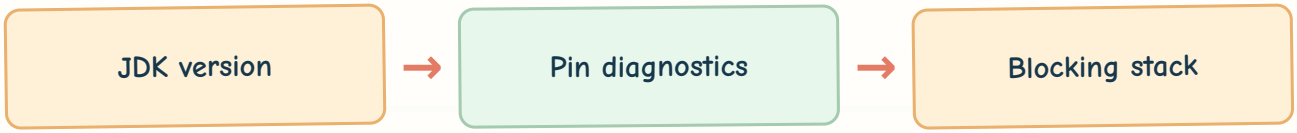
INTERVIEW ANSWER CARD

Q093. How did virtual-thread monitor pinning change?

SHORT ANSWER · SAY THIS FIRST

> **Monitor pinning behavior changed across newer Java releases, so use the specific JDK's rules and diagnostics.**

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

In Java 21, blocking inside synchronized code can pin a carrier. JEP 491 in Java 24 removed the monitor-related pinning limitation; other contexts such as relevant native execution can still matter. Diagnose according to the exact JDK rather than repeating old universal advice.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Java 21: inspect blocking synchronized sections
Java 24+: monitor pinning behavior improved
All versions: preserve bounded downstream access
```

Why it matters: An upgrade can change which recorded symptoms are expected.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What pinning evidence would you collect on the selected JDK?

Answer: Capture JFR virtual-thread pin events or relevant tracing on that JDK, then identify monitor/native blocking call stacks.

> **Interview trap:** 'Never use synchronized with virtual threads' ignores newer runtime behavior and correctness needs.

Source guide p. 56 · Technical references J04-J06, J12-J16, J34-J36

Q094 · Modern Java version traps

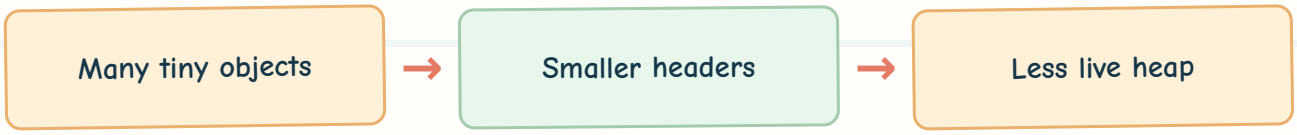
INTERVIEW ANSWER CARD

Q094. What are compact object headers in Java 25?

SHORT ANSWER · SAY THIS FIRST

> Compact object headers in Java 25 can reduce header footprint under the selected supported HotSpot mode.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

JEP 519 made compact object headers a product feature after experimentation in Java 24. This is a HotSpot layout feature, not a new Java object model. Verify support and enablement, then measure footprint and performance; it is not an automatic reduction of every object to one fixed size.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Supporting Java 25: -XX:+UseCompactObjectHeaders
Compare: layout, retained footprint and throughput
Check: other selected VM options and platform support
```

Why it matters: Headers are only one contributor to object size, alongside fields and alignment.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which object-heavy workload might benefit most?

Answer: Object-heavy workloads with many tiny live objects and linked collection nodes may benefit, if CPU and compatibility measurements also improve.

Interview trap: Java 17 measurements cannot validate this Java 25 feature.

Source guide p. 56 · Technical references J04-J06, J12-J16, J34-J36

Q095 · Modern Java version traps

INTERVIEW ANSWER CARD

Q095. How should you evaluate a JDK upgrade for performance?

SHORT ANSWER · SAY THIS FIRST

A JDK upgrade can change GC, JIT and runtime defaults; test correctness and performance together.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Test correctness and compatibility first, then compare startup, steady-state capacity, latency and resources using the same workload. Separate runtime changes from collector/configuration changes where possible. Review release notes and effective options rather than relying on expected speedups.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Test A: old JDK, old compatible config
Test B: new JDK, comparable supported config
Test C: new JDK, deliberate collector/feature change
```

Why it matters: Multiple comparison points help isolate the source of a difference.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What would block an upgrade despite better average throughput?

Answer: Block on incompatible libraries, crashes, data corruption, unacceptable tail latency, higher OOM risk or failed rollback/recovery tests.

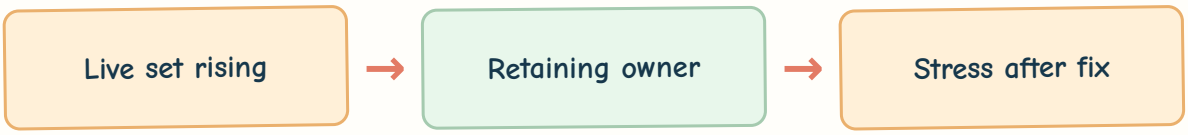
Interview trap: A single faster run after an upgrade is not a reliable attribution.

Source guide p. 56 · Technical references J04-J06, J12-J16, J34-J36

CHAPTER 20 / 20

Senior diagnosis and design scenarios

Investigate with evidence, verify with controlled load.



FIVE ORIGINAL INTERVIEW QUESTIONS

096. A service's heap keeps rising. How do you investigate?

097. RSS rises while Java heap looks stable. What next?

098. p99 is poor but GC pauses are short. What do you do?

099. Would you choose G1 or ZGC for a large Java API?

100. How do you present a credible performance improvement?

> Strong interview answer = what happens, what can fail, and what evidence proves it.

Q096 · Senior diagnosis and design scenarios

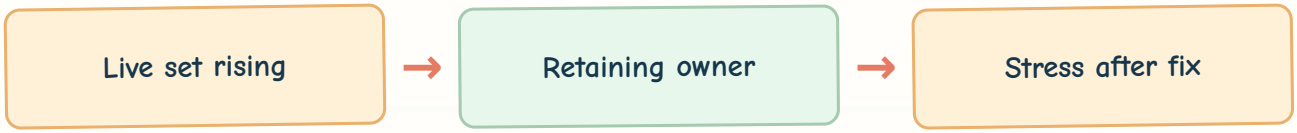
INTERVIEW ANSWER CARD

Q096. A service's heap keeps rising. How do you investigate?

SHORT ANSWER · SAY THIS FIRST

> When heap climbs, compare allocation and post-GC live memory, then identify which owners retain growing object graphs.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Establish workload phases and compare retained/live-set behavior after relevant reclamation, not just instantaneous used heap. Correlate cache, queue and session counts. Capture suitable histograms or dumps, identify retaining roots and test the suspected owner's policy.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Trend: baseline after comparable reclamation rises
Histogram: growing payload objects
Root path: retry queue retains failed requests
Repair: bounded queue, deadlines and safe discard
```

Why it matters: The retaining owner often explains why more heap only delayed failure.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you validate that the repair stops growth under sustained failures?

Answer: Run sustained-load and failure-path tests, verify stable post-GC baselines, bounded caches/queues and safe OOM headroom over time.

> **Interview trap:** A sawtooth graph with temporary growth can be normal.

Source guide p. 57 · Technical references J09-J11, J21-J24, J38

Q097 · Senior diagnosis and design scenarios

INTERVIEW ANSWER CARD

Q097. RSS rises while Java heap looks stable. What next?

SHORT ANSWER · SAY THIS FIRST

Stable heap with rising RSS suggests checking off-heap memory, threads, metaspace and OS accounting.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Inspect direct buffers, threads, tracked native categories, libraries and OS mappings, along with memory-accounting context. Use NMT diffs where enabled and OS evidence where it is not. Compare workload and deployment changes before assigning the growth to a leak.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Heap stable; thread count grows steadily
Inspect: stacks and thread ownership
NMT: tracked Thread category changes
OS: confirm resident/charged-memory trend
```

Why it matters: Unbounded thread creation can consume memory outside the heap.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

Which evidence would distinguish allocator retention from still-growing live native allocations?

Answer: Compare NMT deltas, mapped files, allocator behavior and direct-buffer use; retained native allocations rise differently from reusable allocator arenas.

Interview trap: Stable NMT does not rule out an untracked native library leak.

Source guide p. 58 · Technical references J09-J11, J21-J24, J38

Q098 · Senior diagnosis and design scenarios

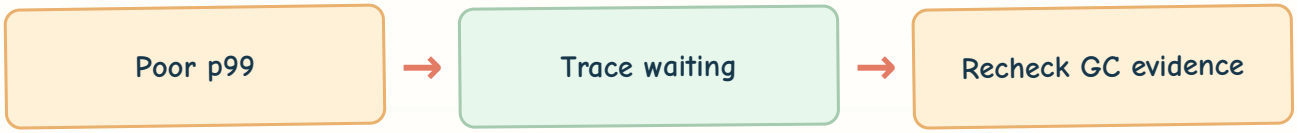
INTERVIEW ANSWER CARD

Q098. p99 is poor but GC pauses are short. What do you do?

SHORT ANSWER · SAY THIS FIRST

Short GC pauses do not rule out request queuing, DB waits, network stalls or CPU contention.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Follow the request timeline through admission, locks, CPU scheduling and dependencies. Inspect allocations and collector stalls only when evidence connects them. Compare endpoint-level traces and saturation rather than assuming all tail latency comes from GC.

JAVA / COMMAND / SCENARIO EXAMPLE

```
GC pauses: short
Trace: 600 ms JDBC acquisition, 15 ms query
Bottleneck: connection availability/admission
Repair: workload and DB budget, not random GC flags
```

Why it matters: Low-pause GC removes one possible contributor rather than every source of delay.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What evidence would change your diagnosis back toward memory management?

Answer: Change direction if allocation stalls, safepoint delays, native-memory pressure or retained-heap trends line up in time with p99 spikes.

Interview trap: Changing to another low-pause collector can leave the dominant wait untouched.

Source guide p. 58 · Technical references J09-J11, J21-J24, J38

Q099 · Senior diagnosis and design scenarios

INTERVIEW ANSWER CARD

Q099. Would you choose G1 or ZGC for a large Java API?

SHORT ANSWER · SAY THIS FIRST

Choose G1 or ZGC by testing the same large Java API, memory/CPU budget and customer SLO.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Start with the service's tail-latency and resource objectives, exact runtime and allocation/live-set profile. Benchmark both where supported, measuring successful throughput, stalls, memory and CPU. Choose the configuration that meets the full objective within the deployment budget.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Constraint: measured p99 and error objective
Budget: fixed CPU and charged-memory limit
Compare: equivalent workload and deployment
Decision: evidence, tradeoff and rollback plan
```

Why it matters: Large heap size alone does not establish a winner.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

What result would make you keep G1 despite shorter ZGC pauses?

Answer: Keep G1 if ZGC uses too much CPU/memory, reduces throughput, causes allocation stalls or does not meaningfully improve user p99.

Interview trap: Collector-specific claims about pause goals are not an application SLA.

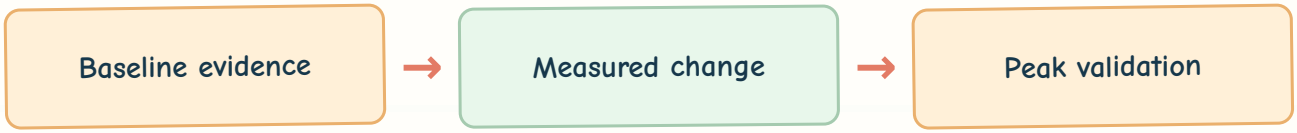
Source guide p. 59 · Technical references J09-J11, J21-J24, J38

Q100. How do you present a credible performance improvement?

SHORT ANSWER · SAY THIS FIRST

> A credible speedup has a controlled baseline, clear mechanism, repeatable measurements and safe rollout.

VISUAL MEMORY SKETCH



PRECISE TECHNICAL EXPLANATION

Describe the initial symptom, measured bottleneck, controlled change, representative test and before/after results with uncertainty. Include errors, resource cost and operational limits. Distinguish a local demonstration from production validation.

JAVA / COMMAND / SCENARIO EXAMPLE

```
Problem -> evidence -> hypothesis -> controlled repair
Result -> latency, useful throughput, CPU and memory
Limits -> workload, runtime, environment and duration
```

Why it matters: A trustworthy report lets another engineer reproduce or challenge the conclusion.

ANSWERED FOLLOW-UP · NO MEMORIZED FILLER

How would you prove the improvement survives peak traffic and a dependency slowdown?

Answer: Repeat under peak and degraded dependencies, report percentile latency, errors, throughput, CPU/RSS and rollback evidence, not just average speed.

Interview trap: 'Changed five flags and it is faster' cannot identify cause or regression risk.

Source guide p. 59 · Technical references J09-J11, J21-J24, J38

VISUAL REFERENCE / 01

JVM memory is more than the heap

Think of the Java process as a shared memory budget, not a single heap graph.



Java heap: object allocation and garbage collection. Xmx is the Java object-heap ceiling, not total resident memory.

Outside the heap: class metadata, compiled code, thread stacks, JIT/GC structures, direct buffers and native libraries use additional memory.

Container: Linux cgroups account for process memory. An external OOM kill can occur without a Java OutOfMemoryError or heap dump.

Example estimate (not an exact RSS equation): 1,024 MiB budget - 512 MiB heap - 320 MiB other use = 192 MiB arithmetic headroom. Measure actual resident/charged memory.

VISUAL REFERENCE / 02

How a young G1 collection works

G1 divides the heap into regions. It evacuates reachable survivors out of chosen regions.



Before: Some objects in young regions are dead; others can still be reached from roots or references in older regions.

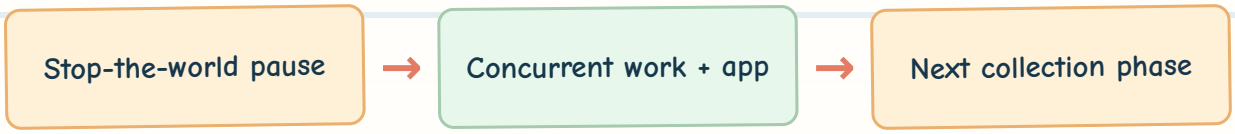
During: G1 pauses application threads, identifies live objects in its collection set and copies them into destination regions.

After: Collected source regions become reusable. Copied bytes, roots and remembered-set work all influence pause duration.

An evacuation failure means that suitable destination space could not be provided. Check live-set pressure, free regions, large allocations and allocation rate.

VISUAL REFERENCE / 03

Collector phases are not synonyms



Parallel: Several GC workers can run together. A parallel GC phase can still stop application threads.

Concurrent: GC overlaps application work and competes for CPU, cache and memory bandwidth.

Low-pause: ZGC and Shenandoah use substantial concurrent work, but can still encounter pauses, stalls or throughput cost.

Version trap: Java 17 ZGC is non-generational; Java 21 added an opt-in generational mode; Java 25 uses generational ZGC. Collector defaults and availability are separate questions.

VISUAL REFERENCE / 04

Three diagnoses, three different fixes

Heap leak
Post-GC live occupancy rises under comparable load; retaining roots show unbounded cache/queue.
Next: Cap ownership and fix retention, then soak-test.

High RSS, stable heap
Inspect direct memory, threads, metaspace, native memory and OS mappings.
Next: Fix the actual native owner; consider container budget.

p99 high, tiny GC pauses
Inspect queuing, DB acquisition, CPU throttling, locks and dependency latency.
Next: Tune the delayed layer rather than guessing new GC flags.

Always correlate a symptom with a timeline, not one screenshot of a heap meter.

SOURCE REFERENCE · ORIGINAL P. 60

Collector comparison

Different GC algorithms optimize throughput, pause behavior and memory overhead.

```
Collector comparison for interviews
COLLECTOR
STARTING USE CASE
TRADEOFF TO EXPLAIN
Serial
Small or constrained
workload
Simple single-worker collection; evaluate pauses as
retained data grows.
Parallel
Throughput-oriented batch
work
Parallel STW work; pause tolerance and CPU
resources matter.
G1
Balanced general service
baseline
Regional evacuation with pause targets and
concurrent work; targets are soft.
ZGC
Low-pause objective on a
supported runtime
Concurrent resource cost and allocation headroom;
modes differ by release.
Shenandoah
Low-pause alternative on
supporting builds
Concurrent evacuation, build availability and
selected mode need verification.
Epsilon
Bounded allocation
experiments
No reclamation; unsuitable as a default long-lived
workload lifecycle.
CMS
Historical only for current
scope
Removed in JDK 14; not a Java 17/21/25 deployment
choice.
No numerical ranking is claimed. Run representative load under the same deployment budget and assess
successful
throughput, tail latency, CPU and memory together.
References: J08-J15, J18, J20, J36, J37, J40
```

> Source-derived reference. Actual runtime version, object lifecycle and deployment limits determine safe conclusions.

SOURCE REFERENCE · ORIGINAL P. 61

Version and mode checklist

Java 17, 21 and 25 differ. Verify actual runtime flags.

```
Java 17, 21 and 25: mode checklist
FEATURE
JAVA 17
JAVA 21
JAVA 25
ZGC
Non-generational
Generational mode
introduced; opt in
explicitly
Generational only after
non-generational
removal in 24
ZGC recipe
UseZGC
UseZGC plus
ZGenerational for
generational mode
UseZGC; omit old
ZGenerational option
Generational
Shenandoah
Not this modern mode
Not the product mode
introduced in 25
Product feature on
supporting builds; mode
selection required
Virtual threads
Not final API
Final feature; monitor
pinning can matter
Monitor-related pinning
improved since 24;
other contexts remain
relevant
Compact object
headers
Not this feature
Not this feature
Product opt-in feature;
inspect support and
effective enablement
Execution here
Local examples ran
Not executed
Not executed
Milestones between columns: generational ZGC became its default mode in Java 23; non-generational
ZGC was removed in 24.
Generational Shenandoah and compact headers were experimental in 24 before product status in 25.
Do not conflate defaults: The default mode of a selected collector is different from the JVM default
collector. Vendor/build/platform availability is a separate requirement.
References: J12-J16, J34-J36
```

Source-derived reference. Actual runtime version, object lifecycle and deployment limits determine safe conclusions.

SOURCE REFERENCE · ORIGINAL P. 62

A real G1 log entry

Understand the event, before/after heap and pause duration.

```
Walkthrough: a real G1 log entry
The following event came from L03 on this environment using G1 with Xms=Xmx=32 MiB. It is one
real local observation from a synthetic bounded workload, not a latency claim for Spring Boot.
[0.076s][info][gc          ] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 14M->2M(32M) 10.862ms
FIELD
READING
Uptime
Time since this JVM began, not a wall-clock date.
GC(0)
Identifier connecting this collection with its associated phase lines.
Pause Young (Normal)
Young collection event and its stated classification.
G1 Evacuation Pause
The event includes relocating live objects in the selected collection set.
Heap values
14M before, 2M after, 32M shown heap capacity. They are log-reported
units.
Duration
10.862 ms reported for this event; subsequent runs can differ.
What this establishes: The workload ran with G1 and produced a young collection with a
before/after heap change. The selected log shows a real event that you can interpret.
What remains unknown: This one event does not establish p99, collector superiority, leak
absence, full live-set size or behavior under container CPU throttling.
Next evidence: Inspect repeated events, old-region trends, allocation/survival patterns and phase
times. Correlate with customer metrics only in a representative application workload.
References: J09, J10, J19
```

> Source-derived reference. Actual runtime version, object lifecycle and deployment limits determine safe conclusions.

SOURCE REFERENCE · ORIGINAL P. 63

Diagnostic command reference

Use the least expensive tool that can answer your question.

```
Run against the intended Java process with suitable ownership/attachment rights and compatible
tools. PID is a placeholder. Minimal images may need a planned diagnostic tool path; do not assume
every command is installed.
QUESTION
COMMAND / EVIDENCE
What launch
configuration?
jcmd PID VM.command_line jcmd PID VM.flags
What is the heap context?
jcmd PID GC.heap_info
Where are threads
waiting?
jcmd PID Thread.print -l
What changed in tracked
native use?
Start with NativeMemoryTracking=summary; then VM.native_memory
baseline and summary.diff.
What happened in the
time window?
jcmd PID JFR.start with a bounded name, duration and filename.
Which classes dominate
shallow bytes?
jcmd PID GC.class_histogram; inspect impact and possible GC.
Who retains the object
graph?
Review jcmd PID help GC.heap_dump and collection impact, then
analyze an appropriately collected dump.
What can this runtime do?
jcmd PID help; command availability varies.
Impact: Collection, dump and high-detail recording options have different operational costs. A
read-like diagnostic command is not always pause-free.
Tool interpretation: Histogram: class totals. Heap graph: retaining ownership. Thread stacks: state
at a moment. JFR: sampled/selected events over time. NMT: supported tracked native allocations.
References: J21-J24
```

> Source-derived reference. Actual runtime version, object lifecycle and deployment limits determine safe conclusions.

SOURCE REFERENCE · ORIGINAL P. 64

Worked diagnosis: rising retained data

Compare retained ownership under comparable workload.

```
Illustrative Spring API incident: memory usage rises during downstream failures. More heap delays
exhaustion, but the recovered baseline still rises across comparable workload windows.
OBSERVATION
HYPOTHESIS / NEXT CHECK
More failed outbound requests
Retry policy may retain requests longer than their useful
lifetime.
Increasing post-reclamation
occupancy
Investigate persistent owners, not only allocation rate.
Histogram shows many request
payloads
Inspect a graph and root path to find the owner.
Path leads to retry queue
Check queue bounds, expiry, cancellation and response
ownership.
Queue accepts unlimited failed
work
Bound admission and deadlines; preserve required durable
work elsewhere.
Repair: Use a bounded retry/admission policy consistent with business durability. Cancel expired
in-memory work, avoid retaining full payloads unnecessarily and define controlled failure or durable
queuing behavior.
Verification: Repeat representative traffic with a deliberately slow dependency. Check that retained
baseline stabilizes, useful successes recover, queue age stays bounded and rejected/expired
operations remain correct.
Interview answer: The collector could reclaim only unreachable objects. The retry owner kept
failed request graphs reachable; tuning GC could not correct that ownership policy.
No production incident or improvement measurement is claimed; this is a design and diagnosis
exercise.
```

> Source-derived reference. Actual runtime version, object lifecycle and deployment limits determine safe conclusions.

SOURCE REFERENCE · ORIGINAL P. 65

Worked diagnosis: latency without long GC

Look at blocked and queued work before changing GC.

```
Illustrative incident: p99 rises after more API replicas are added. GC pause evidence stays short,
but
request traces show long JDBC acquisition waits.
CHECK
REASON
Total possible connections
Replica count multiplies per-process pool maxima.
Acquisition versus execution
Separate waiting for a connection from SQL time.
DB sessions and waits
Determine whether more connections increase useful work or
contention.
Request admission and deadline
Bound the number and lifetime of waiting request graphs.
Autoscaling signals
Scaling JVMs cannot guarantee more database capacity.
Repair candidate: Adjust service concurrency and pool budgets against measured database
capacity. Reduce query/transaction cost where evidence supports it; apply deadlines and explicit
overload behavior.
Verification: At equivalent offered load, compare successful throughput, p99, pool acquisition, DB
waits, allocation and charged memory. Test a database slowdown as well as normal load.
Interview answer: GC was investigated and was not the dominant observed delay. Pool waiting
explained the tail; a different collector would not create database execution capacity.
```

> Source-derived reference. Actual runtime version, object lifecycle and deployment limits determine safe conclusions.

SOURCE REFERENCE · ORIGINAL P. 66

Worked diagnosis: external memory kill

Linux memory enforcement and JVM heap are not identical.

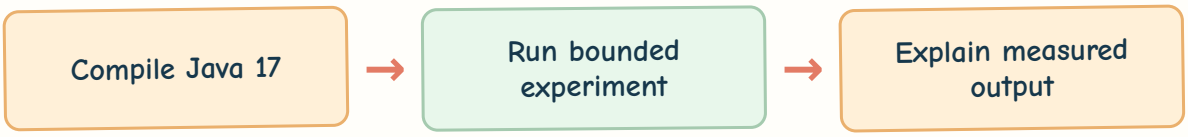
```
Illustrative Kubernetes case: a replica is OOMKilled while Java heap-used is below its configured
maximum. No heap dump appears.
EVIDENCE
QUESTION
Termination and cgroup events
Was the platform memory limit the failing boundary?
Memory graph under peak traffic
What charged-memory peak preceded termination?
Direct-buffer pools and request
sizes
Did concurrent payload buffers grow?
Thread and stack trends
Did platform-thread counts or stack demand increase?
NMT and OS mappings where
available
Which tracked or untracked native categories account for
growth?
Repair candidate: Rebudget heap and measured native contributors. Bound payload buffering,
thread creation and in-flight requests. Choose limit changes only when justified by capacity and
cost.
Verification: Run the same image with the actual pod constraints and representative peak/failure
traffic. Observe charged memory, CPU throttling, GC, errors and useful throughput over a sufficient
window.
Interview answer: Xmx bounded the object heap, while the platform enforced a wider
charged-memory boundary. Heap-dump-on-OOME could not guarantee evidence for the external kill.
```

> Source-derived reference. Actual runtime version, object lifecycle and deployment limits determine safe conclusions.

HANDS-ON PRACTICE

10 JVM exercises

Measure one behavior at a time. Run locally in a disposable directory.



L01–L10: JDK 17+ on Linux/macOS/Windows WSL with Bash; L10 also needs Python 3. Use a full JDK with javac for the scripts.

L09: Deliberate heap exhaustion under a 16 MiB limit in a child process. Never run inside production JVMs.

Toy exercises validate local semantics. They cannot rank GC collectors under production load or establish application p99 guarantees.

HANDS-ON EXERCISE 01 / 10

L01. Inspect the actual runtime and effective ergonomics

Inspect first; do not assume the collector from a remembered JDK default.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
java -version
java -XX:+PrintCommandLineFlags -version
java -XshowSettings:vm -version
```

Expected observation: Identify runtime build, selected collector and estimated heap. These are this process configuration, not evidence about your production container.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 02 / 10

L02. Observe loading versus initialization

The constant expression can be embedded in the caller. Static initialization is not triggered by every class reference.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes InitDemo.java
java -cp classes InitDemo
```

Expected observation: PASS confirms no initialization during the false-initialize Class.forName call or constant read, and one initialization after reading the nonconstant static field.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 03 / 10

L03. Generate a bounded G1 GC log

The loop is an allocation demonstration, not JMH or a service-capacity benchmark. Work and timing vary by runtime and environment.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes GcLoad.java
java -Xms32m -Xmx32m -XX:+UseG1GC \
  '-Xlog:gc*:file=g1.log:uptime,level,tags' \
  -cp classes GcLoad
```

Expected observation: The workload prints PASS and creates g1.log with G1 initialization and collection evidence. Inspect event type, heap before/after and reported duration.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 04 / 10

L04. Run the same allocation source with available collectors

Java 17 ZGC here is non-generational. Elapsed values from one short loop do not rank collectors for production.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes GcLoad.java
for collector in Serial Parallel Z; do
  java -Xms32m -Xmx32m "-XX:+Use${collector}GC" \
    "-Xlog:gc:file=${collector}.log:uptime,level,tags" \
    -cp classes GcLoad
done
```

Expected observation: Each supported collector produces PASS and its own log. On the tested Java 17 build, Serial, Parallel and ZGC all ran. An unsupported collector or platform can reject an option.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 05 / 10

L05. Create and read a JFR recording through its API

This verifies custom recording and reading, not that a CPU profile explains a real application bottleneck.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes JfrDemo.java
java -cp classes JfrDemo
# With a full JDK, optional inspection:
# jfr summary demo.jfr
```

Expected observation: PASS confirms demo.jfr contains exactly ten events named interview.Work. The binary recording is usable even when the standalone jfr command is absent.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 06 / 10

L06. Verify ThreadLocal cleanup on a reused worker

Do not infer automatic context propagation to asynchronous tasks. Scope and ownership must be designed separately.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes ThreadLocalDemo.java
java -cp classes ThreadLocalDemo
```

Expected observation: PASS confirms the second task on the same single-worker executor sees no customer-A context after finally removes it.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 07 / 10

L07. Detect an intentionally isolated deadlock

This is an isolated toy child process. It illustrates detection, not recovery of a real service deadlock.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes DeadlockDemo.java
java -Xmx64m -cp classes DeadlockDemo
```

Expected observation: PASS confirms ThreadMXBean detects the two intentionally deadlocked daemon threads. The JVM exits after the main method; daemon threads do not keep it running.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 08 / 10

L08. Compare bounded and unbounded retention policies

Entry count is not a byte budget when value sizes differ. Both graphs are intentionally small and do not prove heap-dump leak analysis.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes RetentionDemo.java
java -Xmx64m -cp classes RetentionDemo
```

Expected observation: PASS confirms 2,000 versus 100 retained map entries after the same keys are inserted. The bounded example evicts by entry count.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 09 / 10

L09. Reproduce heap exhaustion in a small child process

Catching this expected error is only for demonstration. Continuing a production JVM after memory exhaustion is not presented as a general recovery policy.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
javac --release 17 -d classes OomDemo.java
java -Xms16m -Xmx16m -XX:+UseSerialGC \
  -cp classes OomDemo
```

Expected observation: The isolated child prints PASS after catching its expected heap OOME. Its retained allocations cannot all fit in the selected 16 MiB heap. No heap dump is requested.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

HANDS-ON EXERCISE 10 / 10

L10. Calculate pause fraction and a memory estimate

Concurrent cycle durations must not be counted as stopped time. Estimated memory categories are not a perfect additive RSS model.

TEST SCRIPT PRESERVED FROM SOURCE

```
set -euo pipefail
python3 - <<'PY'
pauses_ms = [8, 12, 10, 20]
window_ms = 10000
fraction = sum(pauses_ms) / window_ms
assert fraction == 0.005
limit_mib = 1024
heap_mib = 512
estimated_other_mib = 320
headroom_mib = limit_mib - heap_mib - estimated_other_mib
assert headroom_mib == 192
print("PASS: illustrative STW fraction =", fraction)
print("Illustrative remaining headroom MiB =", headroom_mib)
PY
```

Expected observation: The illustrative STW fraction is 0.005, or 0.5%, and arithmetic headroom is 192 MiB. These are calculations on example inputs, not measurements.

Verification: Bash syntax and local execution passed using a Java 17 full JDK. Results illustrate the test inputs; they do not prove performance rankings in deployed services.

COMPLETE SOURCES

7 runnable Java files and a JMH template



Complete source: InitDemo, GcLoad, JfrDemo, ThreadLocalDemo, DeadlockDemo, RetentionDemo and OomDemo.

JMH: RepresentationBenchmark.java requires a separate JMH Maven project. It has not been executed; its annotations and commands come from the original guide.

Keep Java 21/25 experiments separate from Java 17 exercise claims. Use identical workloads, hardware and GC modes before comparing results.

COMPLETE TESTED JAVA SOURCE

InitDemo.java

Preserved from the original guide. Compiled with `javac --release 17` and exercised by the associated lab.

```
public class InitDemo {
    static int initialized;
    static class Config {
        static final int CONSTANT = 7;
        static int value = initialize();
        static int initialize() { initialized++; return 9; }
    }
    public static void main(String[] args) throws Exception {
        Class.forName("InitDemo$Config", false,
                InitDemo.class.getClassLoader());
        if (initialized != 0) throw new AssertionError();
        if (Config.CONSTANT != 7) throw new AssertionError();
        if (initialized != 0) throw new AssertionError();
        if (Config.value != 9) throw new AssertionError();
        if (initialized != 1) throw new AssertionError();
        System.out.println("PASS: loading differs from initialization");
    }
}
```

> A reproducible, bounded example - not a production service benchmark.

COMPLETE TESTED JAVA SOURCE

GcLoad.java

Preserved from the original guide. Compiled with `javac --release 17` and exercised by the associated lab.

```
import java.util.ArrayDeque;
public class GcLoad {
    static volatile long checksum;
    public static void main(String[] args) {
        var retained = new ArrayDeque<byte[]>();
        long started = System.nanoTime();
        for (int i = 0; i < 30000; i++) {
            byte[] data = new byte[8192];
            data[0] = (byte) i;
            retained.addLast(data);
            if (retained.size() > 128) {
                checksum += retained.removeFirst()[0];
            }
        }
        if (retained.size() != 128) throw new AssertionError();
        long elapsed = System.nanoTime() - started;
        System.out.println("PASS: bounded allocation workload");
        System.out.println("elapsed_ns=" + elapsed);
        System.out.println("checksum=" + checksum);
    }
}
```

> A reproducible, bounded example - not a production service benchmark.

COMPLETE TESTED JAVA SOURCE

JfrDemo.java

Preserved from the original guide. Compiled with `javac --release 17` and exercised by the associated lab.

```
import java.nio.file.Path;
import jdk.jfr.Event;
import jdk.jfr.Label;
import jdk.jfr.Name;
import jdk.jfr.Recording;
import jdk.jfr.consumer.RecordingFile;
public class JfrDemo {
    @Name("interview.Work")
    @Label("Interview work event")
    static class Work extends Event { int sequence; }
    public static void main(String[] args) throws Exception {
        Path file = Path.of("demo.jfr");
        try (Recording recording = new Recording()) {
            recording.enable(Work.class);
            recording.start();
            for (int i = 0; i < 10; i++) {
                Work event = new Work();
                event.sequence = i;
                event.commit();
            }
            recording.stop();
            recording.dump(file);
        }
        long count = RecordingFile.readAllEvents(file).stream()
            .filter(e -> e.getEventType().getName()
                          .equals("interview.Work"))
            .count();
        if (count != 10) throw new AssertionError(count);
        System.out.println("PASS: ten recorded and readable events");
    }
}
```

> A reproducible, bounded example - not a production service benchmark.

COMPLETE TESTED JAVA SOURCE

ThreadLocalDemo.java

Preserved from the original guide. Compiled with `javac --release 17` and exercised by the associated lab.

```
import java.util.concurrent.Executors;
public class ThreadLocalDemo {
    static final ThreadLocal<String> context = new ThreadLocal<>();
    public static void main(String[] args) throws Exception {
        var pool = Executors.newSingleThreadExecutor();
        try {
            pool.submit(() -> {
                try {
                    context.set("customer-A");
                    if (!"customer-A".equals(context.get()))
                        throw new AssertionError();
                } finally { context.remove(); }
            }).get();
            boolean clean = pool.submit(
                    () -> context.get() == null).get();
            if (!clean) throw new AssertionError();
            System.out.println("PASS: reused worker has no stale context");
        } finally { pool.shutdownNow(); }
    }
}
```

> A reproducible, bounded example - not a production service benchmark.

COMPLETE TESTED JAVA SOURCE

DeadlockDemo.java

Preserved from the original guide. Compiled with `javac --release 17` and exercised by the associated lab.

```
import java.lang.management.ManagementFactory;
import java.util.concurrent.CountDownLatch;
public class DeadlockDemo {
    public static void main(String[] args) throws Exception {
        Object a = new Object(), b = new Object();
        var acquired = new CountDownLatch(2);
        Thread t1 = worker(a, b, acquired);
        Thread t2 = worker(b, a, acquired);
        t1.start(); t2.start();
        var bean = ManagementFactory.getThreadMXBean();
        long[] ids = null;
        for (int i = 0; i < 100 && ids == null; i++) {
            Thread.sleep(10);
            ids = bean.findDeadlockedThreads();
        }
        if (ids == null || ids.length != 2)
            throw new AssertionError("expected two-thread deadlock");
        System.out.println("PASS: two deadlocked daemon threads detected");
    }
    static Thread worker(Object first, Object second,
                         CountDownLatch acquired) {
        Thread t = new Thread(() -> {
            synchronized (first) {
                acquired.countDown();
                try { acquired.await(); }
                catch (InterruptedException e) {
                    Thread.currentThread().interrupt(); return;
                }
                synchronized (second) { }
            }
        });
        t.setDaemon(true); return t;
    }
}
```

> A reproducible, bounded example - not a production service benchmark.

COMPLETE TESTED JAVA SOURCE

RetentionDemo.java

Preserved from the original guide. Compiled with `javac --release 17` and exercised by the associated lab.

```
import java.util.LinkedHashMap;
import java.util.Map;
public class RetentionDemo {
    public static void main(String[] args) {
        Map<Integer, byte[]> unbounded = new LinkedHashMap<>();
        Map<Integer, byte[]> bounded = new LinkedHashMap<>() {
            protected boolean removeEldestEntry(
                    Map.Entry<Integer, byte[]> e) {
                return size() > 100;
            }
        };
        for (int i = 0; i < 2000; i++) {
            unbounded.put(i, new byte[1024]);
            bounded.put(i, new byte[1024]);
        }
        if (unbounded.size() != 2000 || bounded.size() != 100)
            throw new AssertionError();
        System.out.println("PASS: retained entries 2000 versus 100");
    }
}
```

> A reproducible, bounded example - not a production service benchmark.

COMPLETE TESTED JAVA SOURCE

OomDemo.java

Preserved from the original guide. Compiled with `javac --release 17` and exercised by the associated lab.

```
import java.util.ArrayList;
public class OomDemo {
    public static void main(String[] args) {
        var retained = new ArrayList<byte[]>();
        try {
            for (int i = 0; i < 200; i++) {
                byte[] data = new byte[512 * 1024];
                data[0] = 1;
                retained.add(data);
            }
            throw new AssertionError("expected bounded heap exhaustion");
        } catch (OutOfMemoryError expected) {
            retained.clear();
            System.out.println("PASS: isolated child caught heap OOME");
            System.out.println(expected.getMessage());
        }
    }
}
```

> A reproducible, bounded example - not a production service benchmark.

MICROBENCHMARK REFERENCE

RepresentationBenchmark.java

JMH compares boxed and primitive representations after controlled warmup and forks. Requires JMH dependencies; not executed here.

```
package interview;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;
import org.openjdk.jmh.annotations.*;
@State(Scope.Thread)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MICROSECONDS)
@Warmup(iterations=3, time=1)
@Measurement(iterations=5, time=1)
@Fork(2)
public class RepresentationBenchmark {
    @Param({"256", "1024"})
    public int size;
    @Benchmark
    public List<Integer> boxed() {
        List<Integer> result = new ArrayList<>(size);
        for (int i = 0; i < size; i++) result.add(i + 1000);
        return result;
    }
    @Benchmark
    public int[] primitive() {
        int[] result = new int[size];
        for (int i = 0; i < size; i++) result[i] = i + 1000;
        return result;
    }
}
```

Run in JMH project: `mvn clean verify` then `java -jar target/benchmarks.jar RepresentationBenchmark -prof gc`

Do not interpret a microbenchmark as proof of an HTTP-service speedup without realistic traffic and memory checks.

QUICK REVISION

Collector decision sheet

Serial
Few GC worker threads; consider small heaps and tolerant pause goals.

Parallel
High-throughput batch work when pauses are acceptable.

G1
Region-based general-purpose collector balancing throughput and pause targets.

ZGC
Low-pause collector using concurrent work; needs CPU and allocation headroom.

Shenandoah
Low-pause collector with vendor/build/mode availability constraints.

Epsilon
Allocates without collection; use only bounded experiments.

CMS
Removed from JDK 14; not a modern production option.

DIAGNOSTIC COMMANDS

Measure before tuning

Actual Java
`java -version`
JDK vendor and release

Effective flags
`java -XX:+PrintCommandLineFlags -version`
GC/heap ergonomics

System limits
`java -XshowSettings:system -version`
Container resource detection

GC events
`-Xlog:gc*:file=gc.log:uptime,level,tags`
GC phases and timings

JFR
`jcmd PID JFR.start name=diag settings=profile`
Profiles and request-related events

Thread stacks
`jcmd PID Thread.print`
Blocked/waiting threads

Heap histogram
`jcmd PID GC.class_histogram`
Object counts and shallow sizes

NMT summary
`jcmd PID VM.native_memory summary`
HotSpot native allocations

jcmd and its subcommands require a compatible JDK/tooling environment and safe production authorization. Capturing evidence may create overhead.

PRACTICE PLAN

Your 14-day JVM sprint

01–02 JVM areas and class loading; run L01 and L02.

03–04 Object layout, memory budgets and JIT warmup.

05–06 GC reachability, references and ThreadLocals; run L06 and L08.

07–08 Collector comparison and G1 logs; run L03 and L04.

09 ZGC, Shenandoah and Java 17/21/25 differences.

10 OOM causes, heap limits and simple calculations; L09 and L10.

11 JFR, dumps and locks; run L05 and L07.

12 Allocation-aware code and JMH benchmark design.

13 Container JVM budgets and Spring Boot incident diagnosis.

14 Timed mock interview; answer 10 random follow-ups.

Senior approach: name the observed symptom → propose a mechanism → gather evidence → change one variable → retest at peak and during failure.

REFERENCES

Primary JVM sources

Reference links are retained from the source guide. Check vendor documentation for your exact JDK release and build; these links were not revalidated against the live web while generating this edition.

https://docs.oracle.com/javase/specs/jvms/se25/html/jvms-2.html

https://docs.oracle.com/javase/specs/jvms/se25/html/jvms-5.html

https://docs.oracle.com/javase/specs/jls/se25/html/jls-12.html

https://docs.oracle.com/en/java/javase/17/docs/specs/man/java.html

https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html

https://docs.oracle.com/en/java/javase/25/docs/specs/man/java.html

https://docs.oracle.com/en/java/javase/25/gctuning/introduction-garbage-collection-tuning.html

https://docs.oracle.com/en/java/javase/25/gctuning/available-collectors.html

https://docs.oracle.com/en/java/javase/25/gctuning/garbage-first-g1-garbage-collector1.html

https://docs.oracle.com/en/java/javase/25/gctuning/garbage-first-garbage-collector-tuning.html

https://docs.oracle.com/en/java/javase/25/gctuning/z-garbage-collector1.html

https://openjdk.org/jeps/439

https://openjdk.org/jeps/474

https://openjdk.org/jeps/490

https://openjdk.org/jeps/521

https://openjdk.org/jeps/519

https://openjdk.org/jeps/421

https://openjdk.org/jeps/363

https://openjdk.org/jeps/158

https://openjdk.org/jeps/318

https://docs.oracle.com/en/java/javase/25/docs/specs/man/jcmd.html

https://docs.oracle.com/en/java/javase/25/docs/specs/man/jfr.html

https://docs.oracle.com/en/java/javase/25/vm/native-memory-tracking.html

https://docs.oracle.com/en/java/javase/25/troubleshoot/troubleshooting-memory-leaks.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/nio/ByteBuffer.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ThreadLocal.html

https://docs.oracle.com/en/java/javase/17/docs/api/java.management/java/lang/management/ThreadMXBean.html

https://docs.oracle.com/en/java/javase/17/docs/api/jdk.jfr/jdk/jfr/Recording.html

https://github.com/openjdk/jmh

https://github.com/openjdk/jmh/blob/master/jmh-samples/src/main/java/org/openjdk/jmh/samples/JMHSample_08_DeadCode.jav

https://github.com/openjdk/jmh/blob/master/jmh-samples/src/main/java/org/openjdk/jmh/samples/JMHSample_10_ConstantFold.

https://github.com/openjdk/jmh/blob/master/jmh-samples/src/main/java/org/openjdk/jmh/samples/JMHSample_12_Forking.java

https://openjdk.org/jeps/444

https://openjdk.org/jeps/491

https://openjdk.org/jeps/248

https://docs.oracle.com/en/java/javase/25/gctuning/parallel-collector1.html

https://docs.spring.io/spring-boot/reference/actuator/metrics.html

https://docs.spring.io/spring-boot/reference/features/spring-application.html

https://openjdk.org/projects/shenandoah/

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/System.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html

https://docs.oracle.com/javase/specs/jls/se25/html/jls-17.html